Unil.
Université de Lausanne
HEC Lausanne

Session 7: Git from zero — and everything so far

Keep every version of your work, see what changed, share it on GitHub — then review sessions 1–6 for the midterm.

Today has two parts

Part	What	On the midterm?
1 · Git	keep every version of your work; see what changed; share it through GitHub	**No.**
2 · Review	sessions 1–6: types, loops, functions, dicts, files	**Yes — all of it.**

Git is not on the midterm. You learn it today because you need it for your own work, for every assistant that edits your files, and for every project that grows beyond a few files.

If your head is full of the midterm: part 2 starts after the break, and it is exactly the midterm material.

PART 1

Git, from zero

Keep every version of your work, see exactly what changed, and share it. Not on the midterm.

How to read the git slides

Every idea comes **three ways**. The badge in the corner says which.

Badge	What you see	Why
IDEA	a picture	what git keeps, and where
TERMINAL	the command and its real output	what really runs
VS CODE	the buttons	what you will click most days

Every button in VS Code runs a git command. Learn the idea first. Then the command and the button are two ways to say the same thing.

All terminal output on these slides comes from a real run on the course ledger.

IDEA

You already have a way to keep versions

Versions by copying

- `ledger.py`
- `ledger_v2.py`
- `ledger_v2_fixed.py`
- `ledger_final.py`
- `ledger_final_REAL.py`
- `ledger_final_REAL (1).py`

Which one works? What changed between them? Why?
The names cannot tell you.

Versions with git

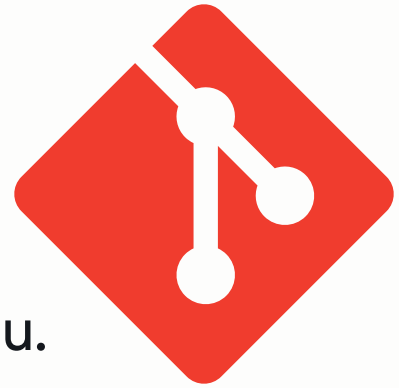
`ledger.py`

one file — and its history:

- `cd81ebf` session 6: read csv, write report
- `3c6342e` session 5: positions as dicts
- `003891f` session 4: functions
- `9de8ebc` session 3: loop and total
- `ee7054a` session 2: value one position
- `916acb4` session 1: first run

The names cannot say what changed, when, or why — or whether it was you or the assistant you asked.

Version control: the idea



A **version control system** keeps every saved version of a folder for you.

- **One file name.** `ledger.py` stays `ledger.py`.
- **Every version is kept**, with who saved it, when, and a sentence saying why.
- You can **compare** any two versions, line by line.
- You can **go back** to any version.

Git is the version control system almost everyone uses. Created in 2005 for the Linux project; now the standard.

Git logo by Jason Long, CC BY 3.0

Why you need it now

1. **Assistants edit your files.** A harness changes five lines when you asked for one. Git shows you exactly which lines changed — and puts them back if you say no. (You will see this today.)
2. **Your project needs a memory.** More on the next slide.
3. **Every team uses it.** Data teams, banks, research labs, software companies. "Send me the link to the repository" is a normal request at work.

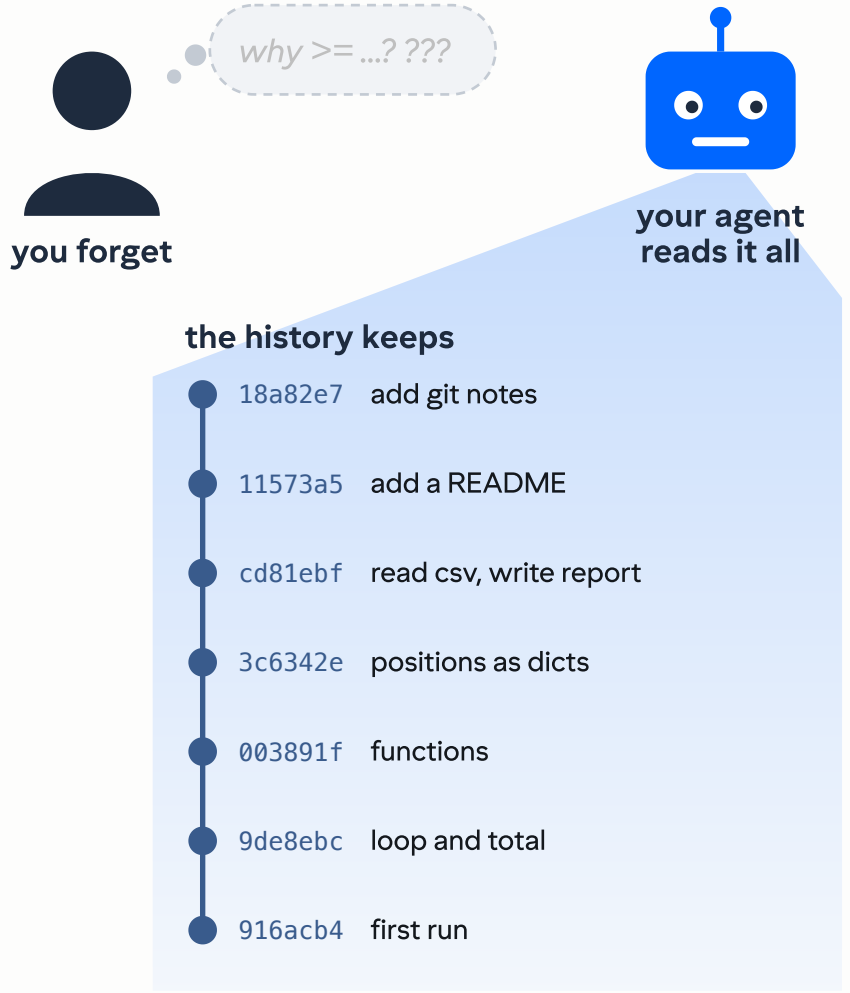
Git is your project's memory

A small script fits in your head. A project does not. After a few weeks you will not remember **why** the fee rule says `>=`, or **when** UBSG joined the ledger.

You forget. The history does not. Every commit keeps what changed, when, and — in its message — why.

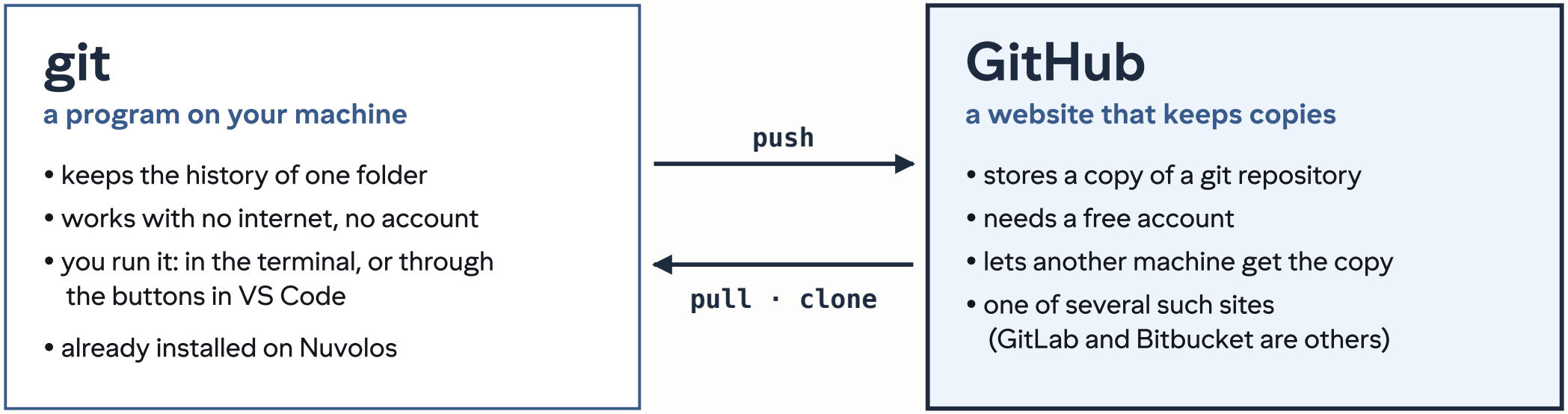
An AI agent starts every session knowing nothing about your project. It cannot remember last week's conversation. But it can read the history — `git log`, `git show`, `git diff` — in seconds, and pick up where you left off. Even what **you** have forgotten.

The bigger the project, the more this matters. **Your commit messages are notes to your future self — and to your agent.**



IDEA

Git and GitHub are not the same thing

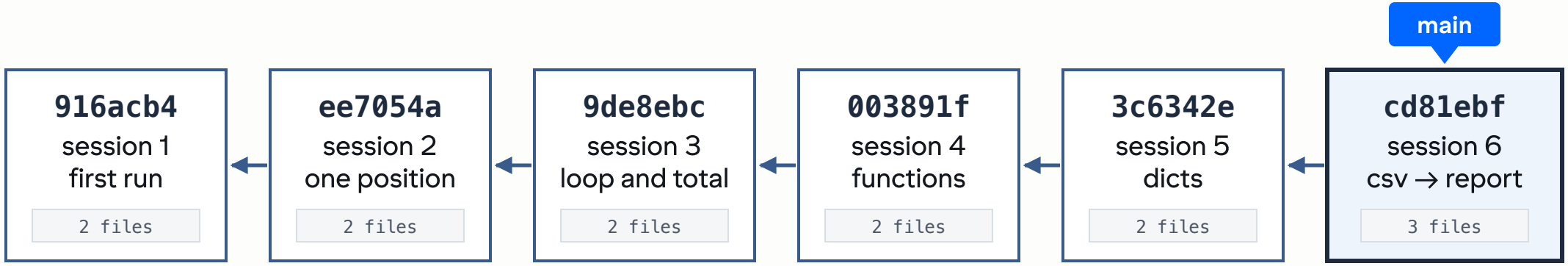


You can use git all your life without GitHub. You cannot use GitHub without git.

GitHub is one provider among several. GitLab and **Bitbucket** do the same job, and many companies, banks and universities run **their own git servers**, so their code never leaves the building. The git commands are the same everywhere; only the address changes.

IDEA

A commit is a snapshot of the whole folder



Each box is the whole folder, frozen. Each arrow says "I came after this one". The name on top is the hash.

When you **commit**, git takes a picture of every file in the folder, exactly as it is. You decide when. Each picture is kept for good.

`main` is a **name** that points at the newest commit; it moves forward with each new one. This course uses only `main`. (Other names — **branches** — exist. We do not need them.)

IDEA

What one commit holds

commit cd81ebf36c22047fb80d81739bf2a97b8c25b27b

snapshot	`ledger.py` `ledger.csv` `.gitignore`
message	session 6: read csv, write report
author	Student <student@unil.ch>
date	Mon Oct 26 16:30:00 2026 +0100
parent	`3c6342e` (session 5: positions as dicts)

The name on top is computed from everything below it.

The **snapshot** is every file in the folder, as it was — not just the changes.

The **message** says why. You write it.

The **author** comes from git config (you set it once per machine).

The **parent** is the commit before. That link makes the chain.

TERMINAL

The name of a commit is a fingerprint

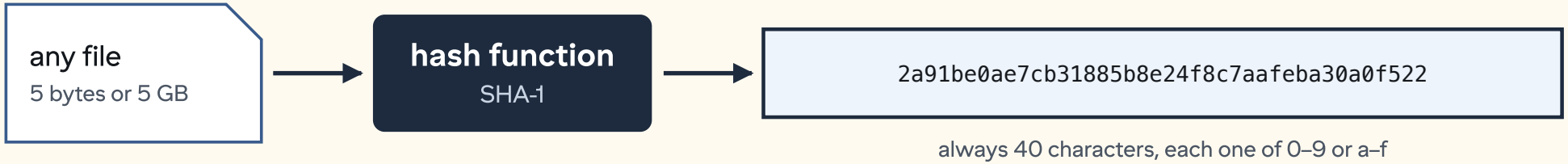
```
$ git hash-object ledger.csv
2a91be0ae7cb31885b8e24f8c7aafeba30a0f522   same file → same hash
$ git hash-object ledger.csv
2a91be0ae7cb31885b8e24f8c7aafeba30a0f522   every time
(changed 190.50 to 190.51 — one character)
$ git hash-object ledger.csv
622dfe9e846e9d2a3eba69995072c2508adb86c2   one character → a new hash
```

- A **hash** is a 40-character name that git **computes from the contents**. Same contents → same hash. Always, on every computer.
- Change one character → a completely different hash.
- Commits are named the same way. So a hash names **one exact version**, and nobody can change that version without its name changing too.
- You never type all 40 characters: the **first 7 are enough**, like `cd81ebf`. Your own hashes will differ: your name, your dates, your files.

IDEA

+ GOOD TO KNOW

What is a hash?



A **hash function** reads any input and returns a short name of fixed length. Git's is called **SHA-1**. Three properties make it useful:

1. **Same input → same hash**, on every computer, forever.
2. **Change one character → a completely different hash.** (Previous slide.)
3. **One way only.** From the hash you cannot rebuild the file.

Why is it unique? Strictly, it is not: there are more possible files than hashes. But there are $16^{40} \approx 1.5 \times 10^{48}$ possible hashes. In a repository with ten million files and commits, the chance that any two share a hash is about 3 in 10^{35} — less likely than winning the Swiss Lotto jackpot four weeks in a row. Unique in practice.

TERMINAL

Snapshots are cheap

```
$ git ls-tree 003891f
100644 blob 20f1eb8b0e01ae8bc56fa5fc40ce28da00d9ef84    ledger.csv
100644 blob 02e38a143a3c07d96d850ff3ce0d93ab8def9a1d    ledger.py
$ git ls-tree cd81ebf
100644 blob 32d27f60b3c8e086b33e79c7da8c71db031a7406    .gitignore
100644 blob 20f1eb8b0e01ae8bc56fa5fc40ce28da00d9ef84    ledger.csv   same hash: stored once
100644 blob dc19e8c87d97f4e38147a3aa8df78e0f6cadf299    ledger.py
```

"Every file in every commit" sounds huge. It is not.

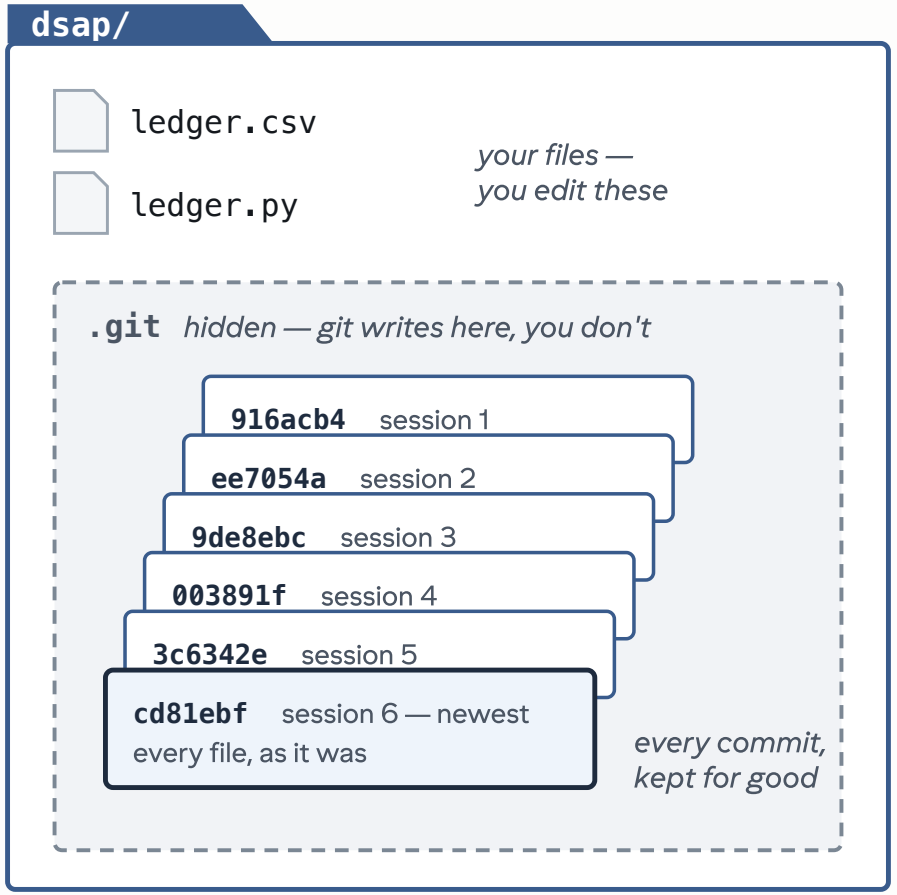
- `ledger.csv` has **the same hash** in session 4 and session 6. Same contents → same hash → **git keeps one copy**, and both snapshots point to it.
- Only a file that changed costs new space — and git compresses everything it keeps.
- So a commit costs almost nothing. The Linux project keeps over a million commits this way. **Commit often.**

IDEA

A repository = a folder + its history

A **repository** (or **repo**) is a normal folder, plus a hidden folder called `.git` that holds every commit.

- Your files stay where they are. You edit them as before.
- Git only reads and writes inside `.git`.
- **Never delete `.git`.** Delete it and the history is gone. The files stay; every old version is lost.
- **One repository per project.** Your `dsap` folder is one. Do not start a second one inside it.



TERMINAL

First contact: is git there?

```
$ git --version
git version 2.55.0
$ git init
Initialized empty Git repository in /files/dsap/.git/
$ ls -a
.
..
.git
ledger.csv
ledger.py
```

the history will live here

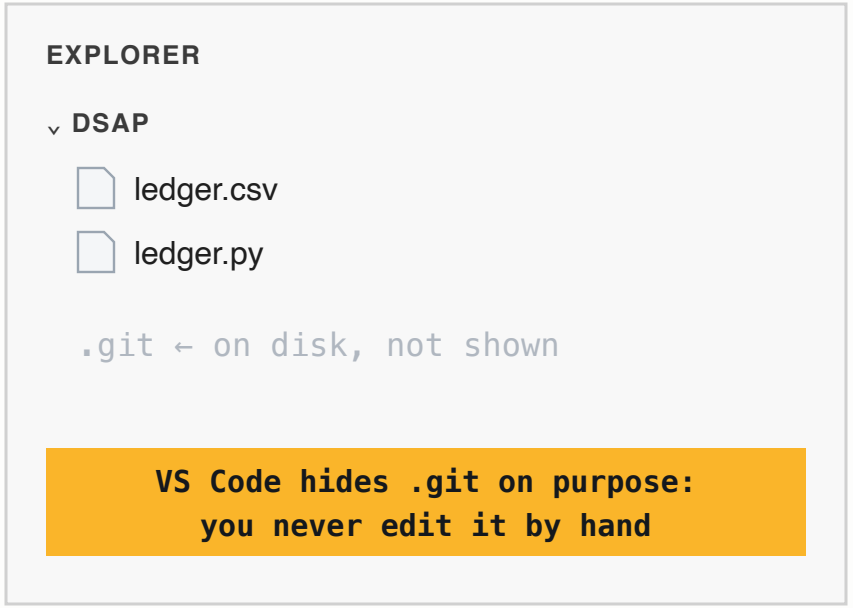
hidden: its name starts with a dot

- Git is **already installed on Nuvolos**. Nothing to install. (Your version number may differ. That is fine.)
- `git init` turns the folder you stand in into a repository. You do it **once per project**.
- `ls -a` lists hidden names too: the ones that start with a dot.

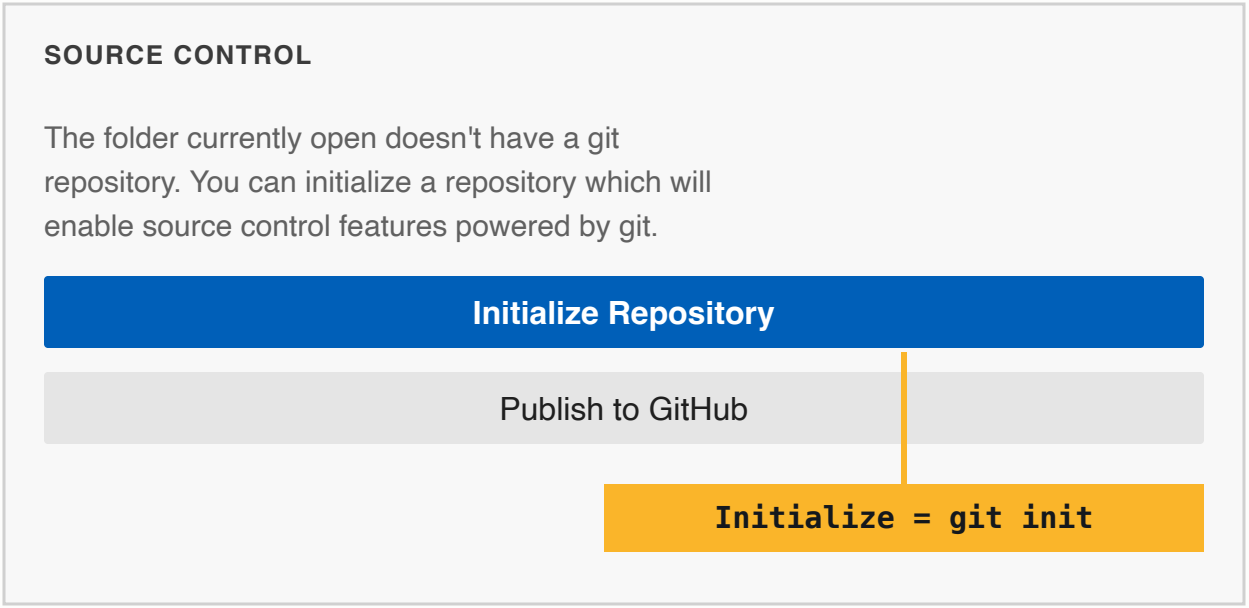
VS CODE

The same, in VS Code

Explorer: the files you see



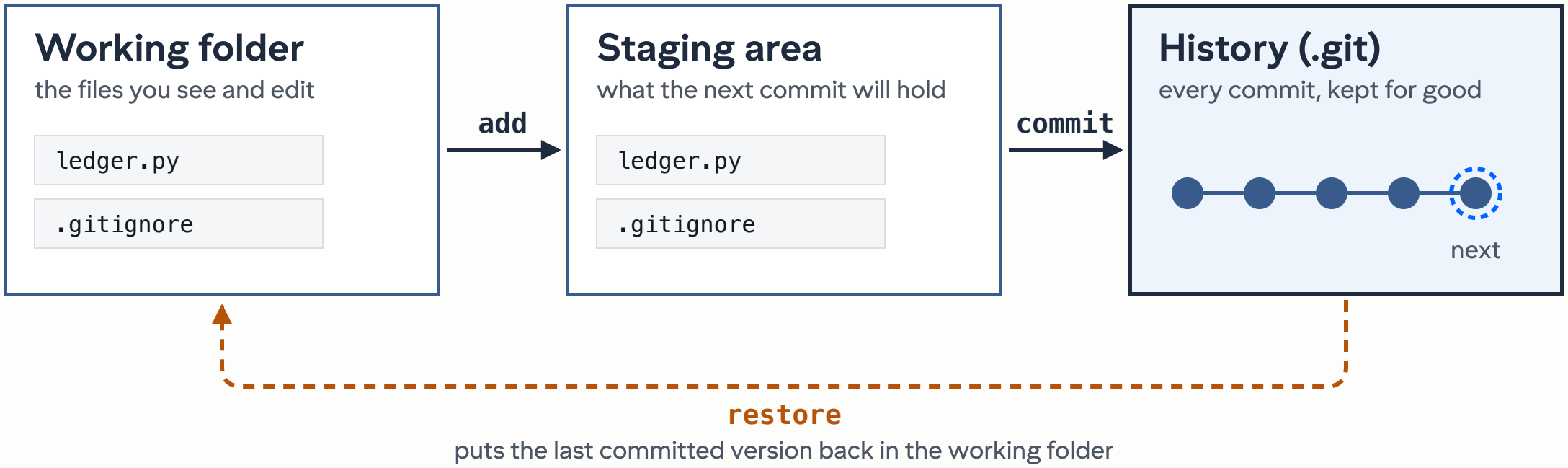
Source Control, before the first git init



Hidden folders again — like the file extensions Windows hides (session 1). Hidden is not gone.

IDEA

A change lives in one of three places



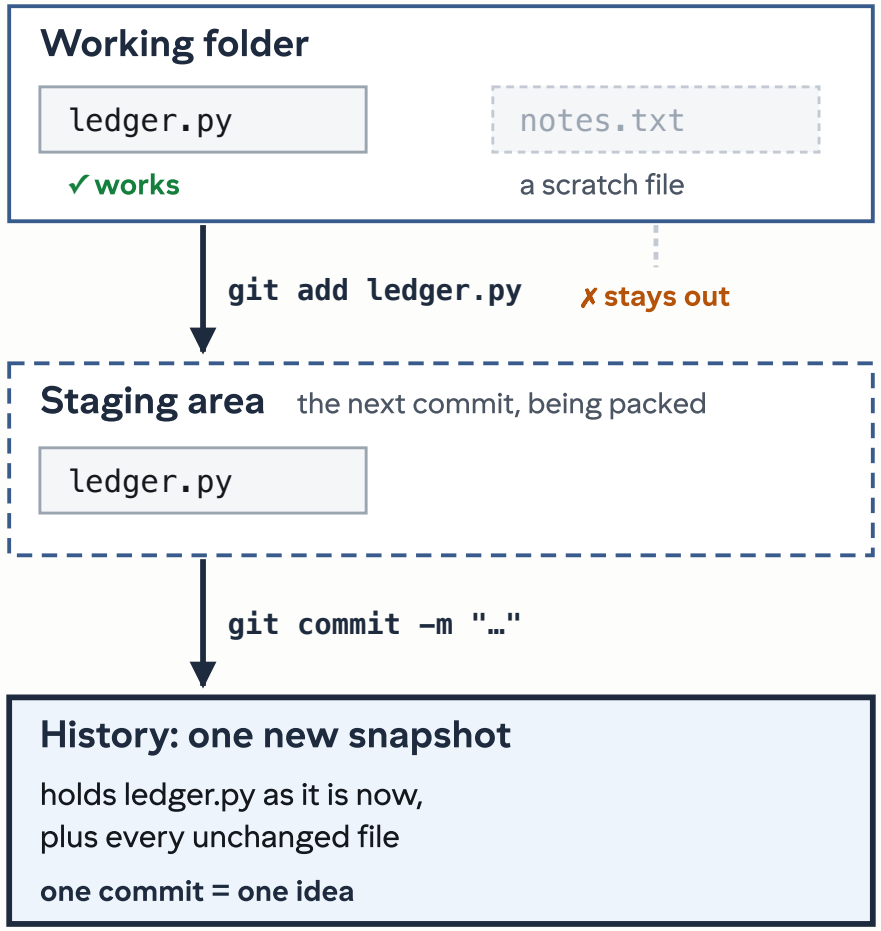
IDEA

Why a middle step?

The **staging area** is where you choose what the next snapshot holds.

- You changed `ledger.py` and it works. You also have `notes.txt`, a scratch file. The snapshot should hold the first, not the second.
- `add` = "this goes in the next commit". Nothing is saved yet.
- `commit` = "save what I added, as one snapshot, with this message".

One commit should be **one idea** — a message you can write in one line. Staging is how you build it.



Every file is in one of four states

State	Meaning	`git status --short`	VS Code
untracked	new; git has never saved it	`??`	**U**
modified	saved before; changed since	`M`	**M**
staged	added; goes into the next commit	`M` or `A`	under **Staged Changes**
committed	same as the last snapshot	(not listed)	(not listed)

`git status` is the question you ask most. Ask it before every `add` and every `commit`.

TERMINAL

`git status` on a brand-new repository

```
$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        ledger.csv
        ledger.py

nothing added to commit but untracked files present (use "git add" to track)
```

never saved: untracked

Read it top to bottom: which name (`main`), how many commits (none), and two files git has never saved. The lines in brackets tell you **the next command**.

TERMINAL

`git add`: choose what goes in

```
$ git add ledger.py ledger.csv
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   ledger.csv
        new file:   ledger.py
```

prints nothing: success

staged

`add` prints nothing when it works. That is normal: silence means success. Ask `git status` to see the result: both files are now **to be committed** — staged.

TERMINAL

The first commit on a new machine fails

```
$ git commit -m "session 1: first run"
Author identity unknown

*** Please tell me who you are.

Run

  git config --global user.email "you@example.com"
  git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.

fatal: no email was given and auto-detection is disabled
```

type these two lines, with your details

Every commit records an **author**. On a new machine git does not know you yet — so it stops, and says exactly what to type.

TERMINAL

Tell git who you are — once per machine

```
$ git config --global user.name "Student"
$ git config --global user.email "student@unil.ch"
$ git commit -m "session 1: first run"
[main (root-commit) 916acb4] session 1: first run
 2 files changed, 9 insertions(+)
 create mode 100644 ledger.csv
 create mode 100644 ledger.py
$ git status
On branch main
nothing to commit, working tree clean
```

the new commit's name

nothing unsaved

- Use **your real name** and **your UNIL email**. GitHub will show them.
- `--global` = for every repository on this machine.
- `[main (root-commit) 916acb4]`: on `main`, the very first commit (it has no parent), named `916acb4`.
- `working tree clean` = nothing changed since the last snapshot.

VS CODE

The same message, from the buttons

First commit on a new machine




```
terminal: *** Please tell me who you are.
```

VS Code shows a short message; the terminal shows the full one. The fix is the same: the two `git config` lines.

A good commit message

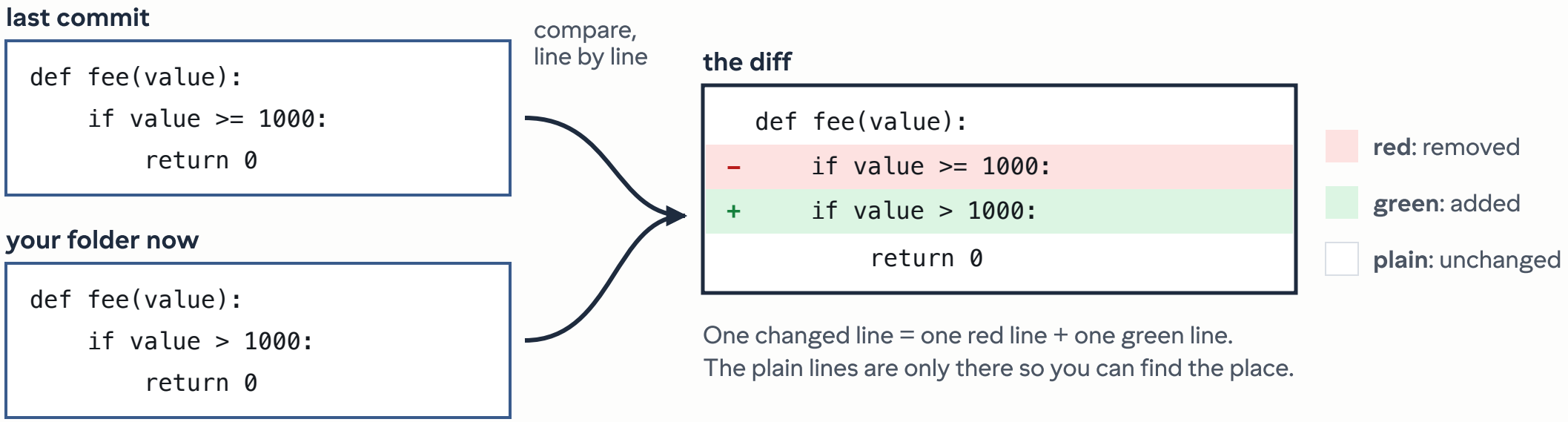
The message is the **why**, for a human — often you, three weeks later.

Good	Bad
`session 2: value one position`	`update`
`fix fee rule: 1000 or more pays no fee`	`fix`
`read the ledger from ledger.csv`	`asdf`
`add a README`	`changes`

- One line. Starts with what the commit does.
- If you cannot say it in one line, it is probably two commits.

IDEA

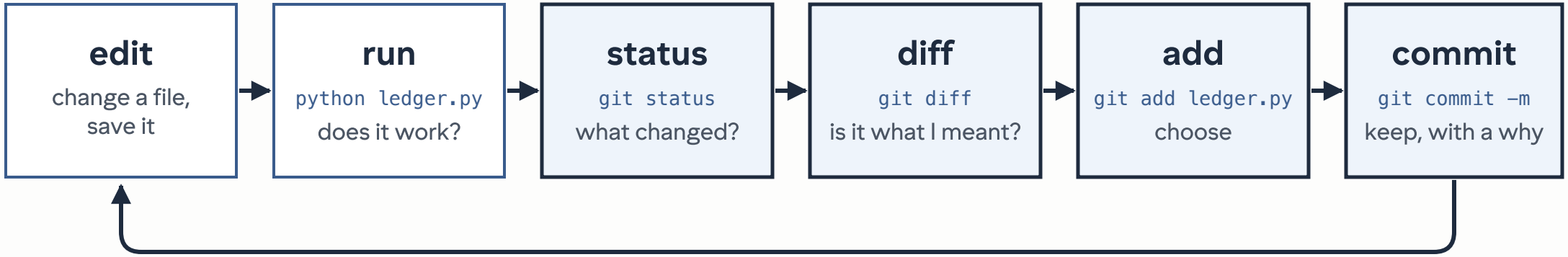
What is a diff?



A **diff** is the difference between two versions of a file, line by line, in color. You will see diffs everywhere: **VS Code** (click a changed file) · **coding agents** (after every edit) · **GitHub** (every commit) · **the terminal** (`git diff`).

IDEA

The everyday loop



and again — one small commit for each thing that works

Four git commands, every day. Everything else is for special days.

TERMINAL

The loop, session 2

```
$ git status --short
 M ledger.py
$ git diff --stat
 ledger.py | 11 ++++++-----
 1 file changed, 6 insertions(+), 5 deletions(-)
$ git add ledger.py
$ git commit -m "session 2: value one position"
[main ee7054a] session 2: value one position
 1 file changed, 6 insertions(+), 5 deletions(-)
```

modified, not yet staged

lines added and removed

- `--short` : one line per file. `M` = modified, not yet staged.
- `git diff --stat` : which files changed, and how much. `6 insertions(+), 5 deletions(-)` counts lines.

TERMINAL

The loop, sessions 3 and 4 (5 is the same)

```
$ git status --short
 M ledger.csv
 M ledger.py
$ git add .
$ git commit -m "session 3: loop and total"
[main 9de8ebc] session 3: loop and total
 2 files changed, 15 insertions(+), 6 deletions(-)
$ git status --short
 M ledger.py
$ git add .
$ git commit -m "session 4: functions"
[main 003891f] session 4: functions
 1 file changed, 11 insertions(+), 7 deletions(-)
```

every change in this folder

`git add .` means "add every change in this folder" (`.` is the folder you stand in). Handy — and dangerous. Next slide.

TERMINAL

Session 6: three changes, one of them scratch

```
$ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   ledger.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        notes.txt   scratch: must not go in

no changes added to commit (use "git add" and/or "git commit -a")
```

`ledger.py` changed. `.gitignore` is new (two slides from now). `notes.txt` is a scratch file. **It should not go in.**

TERMINAL

`git add .` took too much — take one back

```
$ git add .
$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   .gitignore
        modified:   ledger.py
        new file:   notes.txt      oops
$ git restore --staged notes.txt   un-add
$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   .gitignore
        modified:   ledger.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        notes.txt                  back outside
```

`git restore --staged` = **un-add**. The file itself is untouched.

TERMINAL

Commit what you chose

```
$ git commit -m "session 6: read csv, write report"
[main cd81ebf] session 6: read csv, write report
 2 files changed, 12 insertions(+), 9 deletions(-)   ← the two you chose
 create mode 100644 .gitignore
$ python ledger.py
wrote report.txt
$ git status --short
?? notes.txt   ← still outside
```

Two files went in; `notes.txt` stayed out (`??` = untracked).

But the script just wrote `report.txt` — why does `git status` not list it?

`.gitignore`: files git must never keep

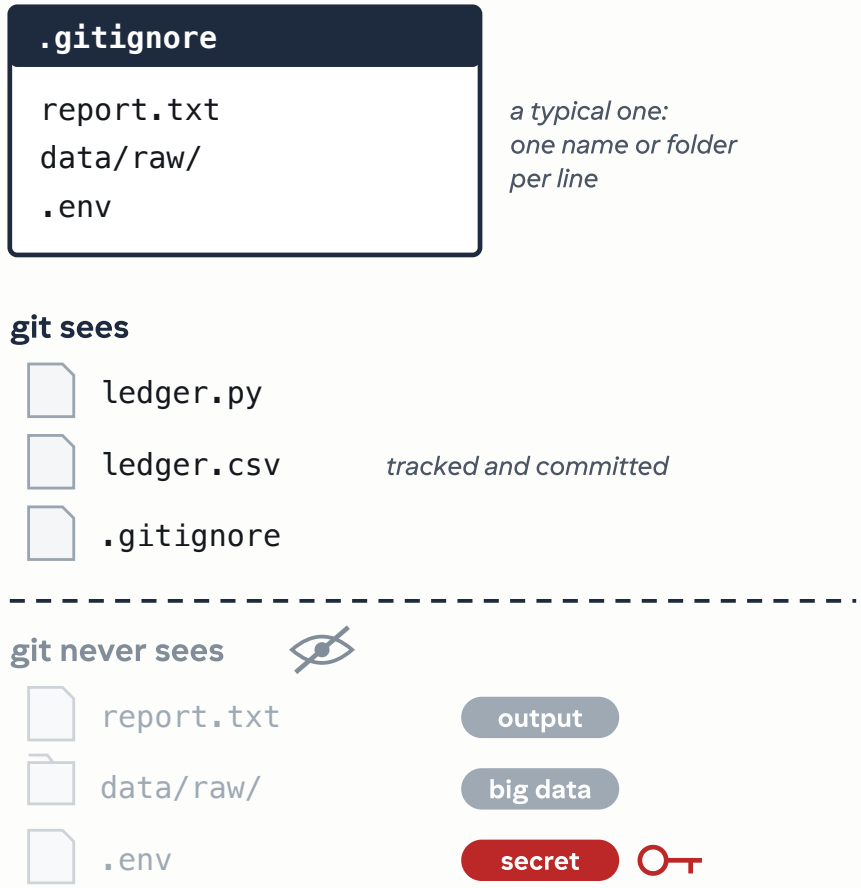
`.gitignore` is a text file with one name (or pattern) per line. Ours holds one line:

```
report.txt
```

Git then acts as if `report.txt` does not exist.

Ignore:

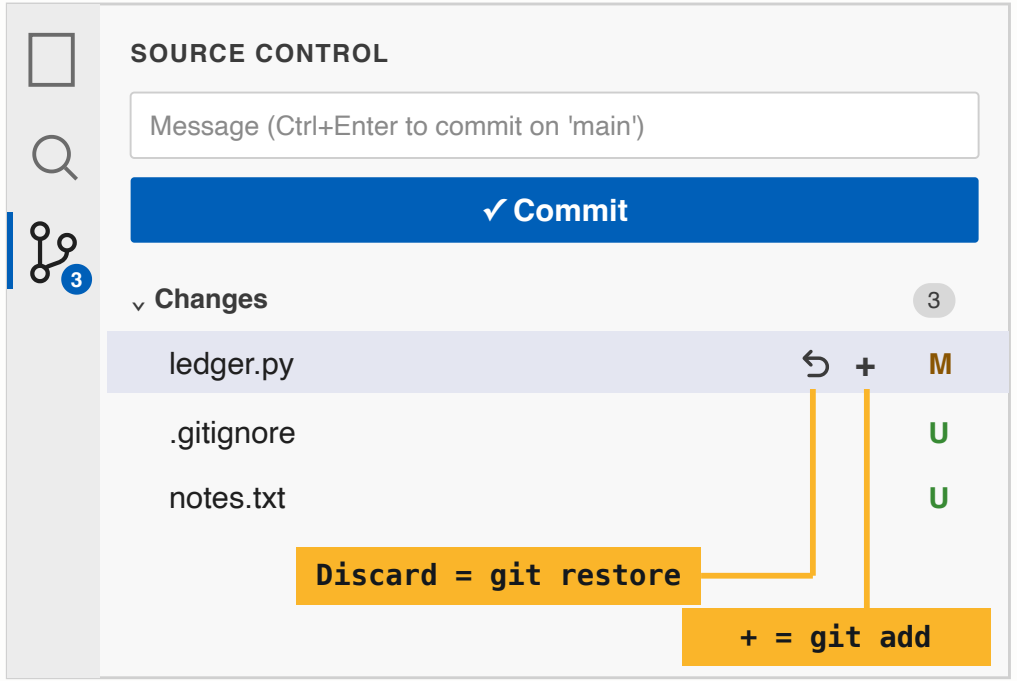
- **outputs** a script re-creates (`report.txt`, figures)
- **big data files** you can download again
- **secrets** — passwords, keys. **Never commit a secret.** Deleting it later does not remove it from the history.



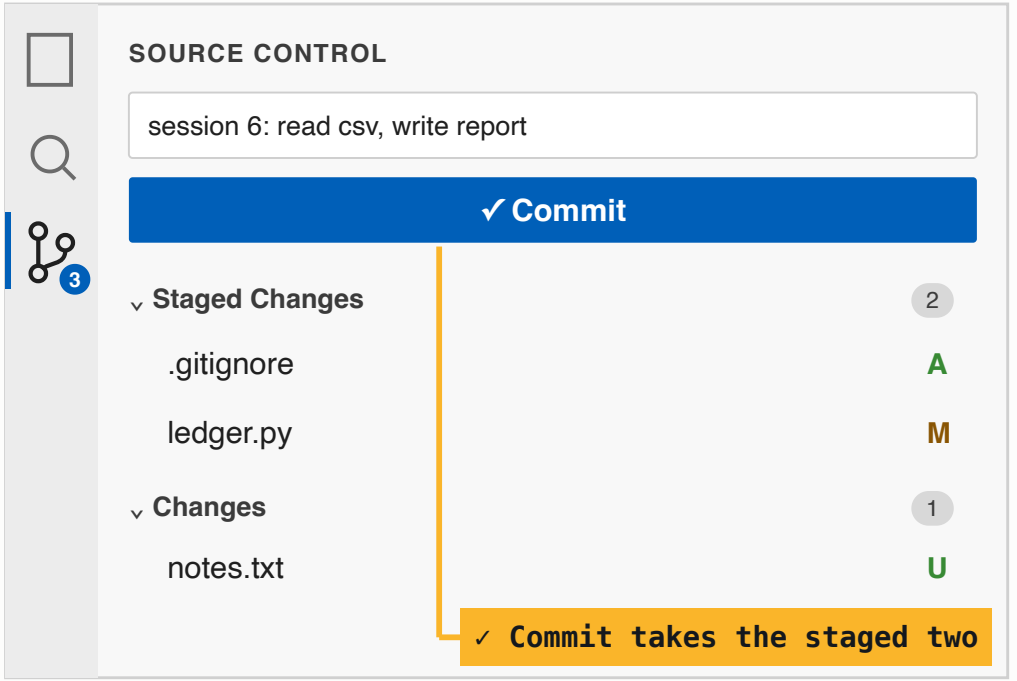
VS CODE

The three places, in VS Code

① Three changes, nothing staged yet



② After pressing + on two of them



The left bar icon with the number is **Source Control**. The number counts changed files.

TERMINAL

Reading history: `git log --oneline`

```
$ git log --oneline
cd81ebf session 6: read csv, write report
3c6342e session 5: positions as dicts
003891f session 4: functions
9de8ebc session 3: loop and total
ee7054a session 2: value one position
916acb4 session 1: first run
```

newest first

the very first commit

Newest first. One line per commit: the short hash, then the message. Good messages make this list worth reading.

TERMINAL

The full record: `git log`

```
$ git log -2
commit cd81ebf36c22047fb80d81739bf2a97b8c25b27b
Author: Student <student@unil.ch>
Date:   Mon Oct 26 16:30:00 2026 +0100

    session 6: read csv, write report

commit 3c6342ebc8ca710d70a7508fc130c53e1d2a7b61
Author: Student <student@unil.ch>
Date:   Mon Oct 19 16:30:00 2026 +0200

    session 5: positions as dicts
```

the full name

from git config

`-2` = the newest two. Each record: the full hash, the author, the date, the message. (Without `-2` it shows all; press `q` to leave if the list is long.)

TERMINAL

One commit up close: `git show`

```
$ git show --stat 9de8ebc
commit 9de8ebc472b5b34ae55fe9b4197f011f712d1813
Author: Student <student@unil.ch>
Date:   Mon Oct 5 16:30:00 2026 +0200

    session 3: loop and total

 ledger.csv |  1 +
 ledger.py  | 20 ++++++++++++++------
 2 files changed, 15 insertions(+), 6 deletions(-)
```

one line added

`git show <hash>` shows one commit. `--stat` shows only which files changed, not every line.

TERMINAL

When did UBSG join the ledger?

```
$ git log --oneline -- ledger.csv
9de8ebc session 3: loop and total   ← touched here
916acb4 session 1: first run
$ git diff 916acb4 9de8ebc -- ledger.csv
diff --git a/ledger.csv b/ledger.csv
index 2a91be0..20f1eb8 100644
--- a/ledger.csv
+++ b/ledger.csv
@@ -2,3 +2,4 @@ ticker,shares,price
 AAPL,10,190.50
 NESN,4,102.30
 VOO,2,512.00
+UBSG,40,25.00   ← the added row
```

`-- ledger.csv` = only this file. Two commits touched it. The diff between them has one line with `+`: the row that was added.

IDEA

Undo: what git can and cannot give back

You want to	Command	Safe?
throw away an edit you have not committed	`git restore <file>`	the edit is **gone** — no undo for the undo
un-add a file (keep the edit)	`git restore --staged <file>`	yes
see what an old commit changed	`git show <hash>`	yes — it only reads

Git can only give back what you committed. Work you never committed is not in `.git`. If you delete it, or `restore` over it, it is gone.

So: **commit before anything risky** — before a big change, and before any assistant edits your files.

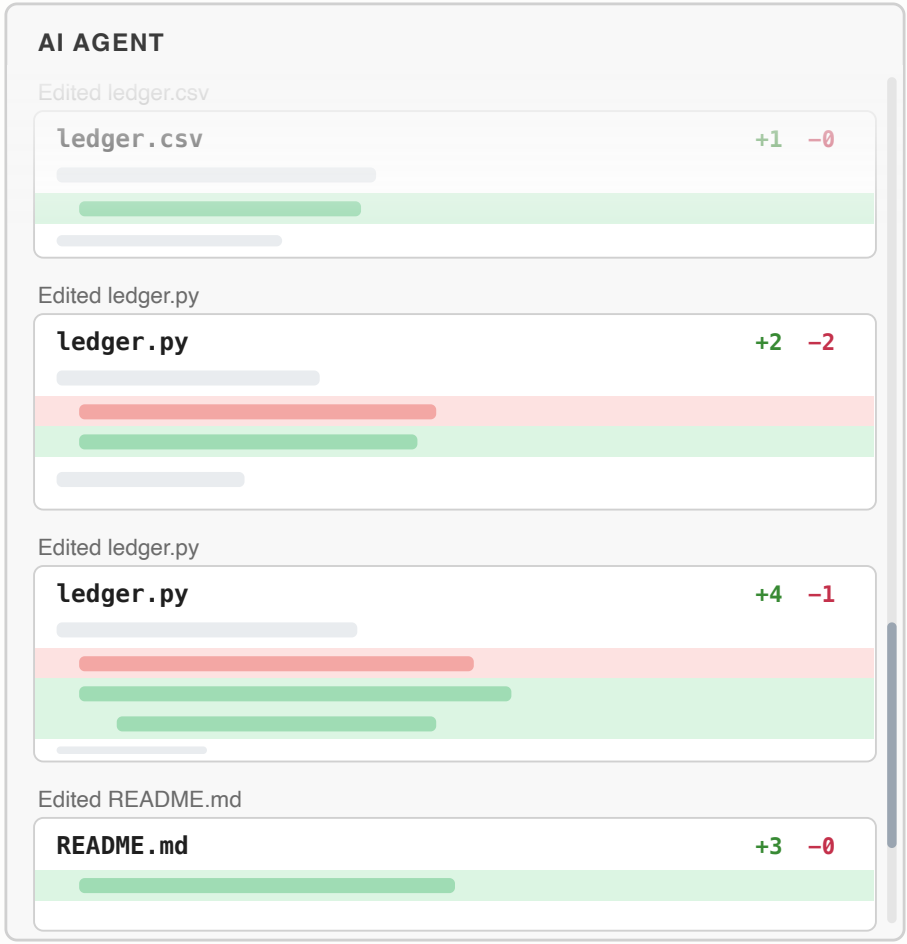
Coding agents show you diffs — lots of them

A coding agent edits your files itself. After **every** edit it shows you a **colored diff**, by default — you do not have to ask.

In a long task that is dozens of diffs, scrolling past quickly. You will not read them character by character. You learn to **scan**:

- **Which file?** And how big is the change?
- **How many colored blocks?** More than you expected?
- **Each red–green pair:** did I ask for this?

Anything you did not ask for: **stop**, and read that part properly. This is a skill, and it gets fast with practice.



An assistant edits your code

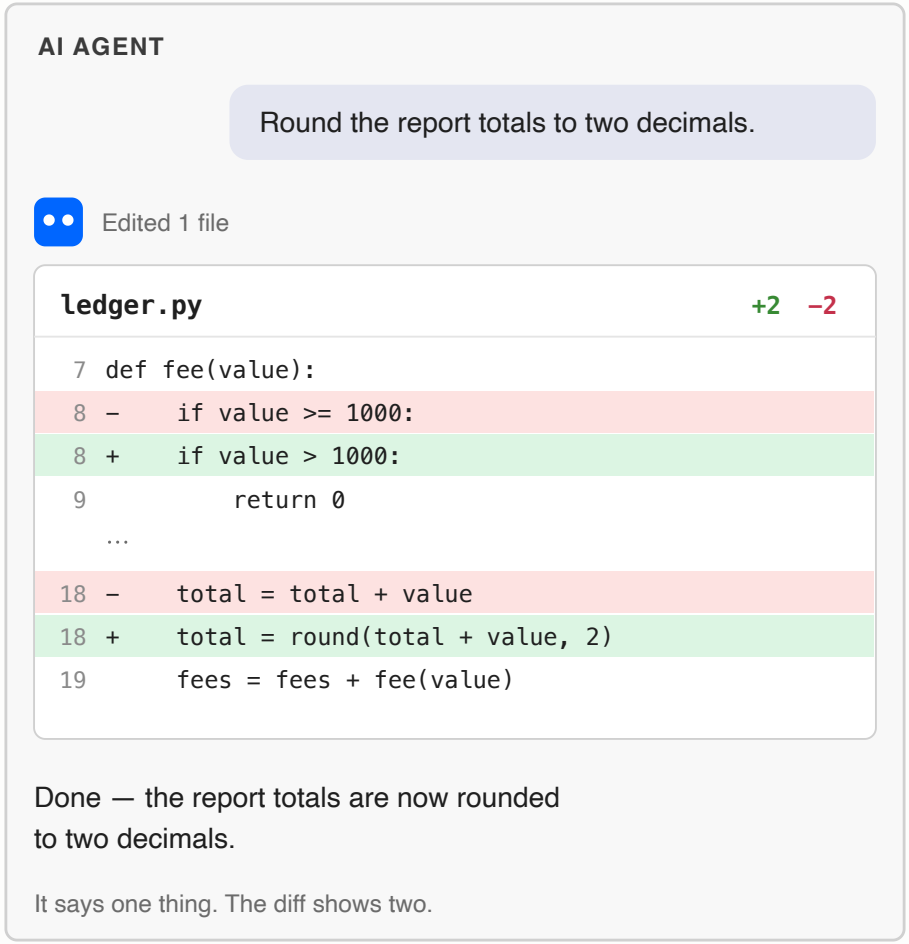
Your repository: session 6 committed, working tree clean.

You ask an agent (an AI tool that edits your files itself):

> "Round the report totals to two decimals."

It edits the file, **shows you a diff** of what it changed, and answers: "Done." **Scan the diff.** Here it shows two changes.

But it is the agent's own report, of this one step. **`git diff` is the independent record:** every change since your last commit, in every file — including earlier steps, and files the agent did not mention.



TERMINAL

Ask git: what changed?

```
$ git status --short
 M ledger.py
$ git diff -U1
diff --git a/ledger.py b/ledger.py
index dc19e8c..5ab8f57 100644
--- a/ledger.py
+++ b/ledger.py
@@ -7,3 +7,3 @@ def position_value(shares, price):
 def fee(value):
-    if value >= 1000:
+    if value > 1000:
         return 0
@@ -17,3 +17,3 @@ for p in positions:
     value = position_value(int(p["shares"]), float(p["price"]))
-    total = total + value
+    total = round(total + value, 2)
     fees = fees + fee(value)
```

`git status --short`: one file changed. `git diff`: exactly how, in the same colors. (`-U1`: one unchanged line of context.)

Reading a diff, fast

ledger.py +2 −2

```
 7  def fee(value):
 8  -     if value >= 1000:
 8  +     if value > 1000:
 9            return 0
    ··· 7 unchanged lines
17        value = position_value(…)
18  -     total = total + value
18  +     total = round(total + value, 2)
19        fees = fees + fee(value)
```

1. **Which file?** And how much: +2 −2 is small. +200 is not.
2. **How many colored blocks?** Two here. You asked for one change.
3. **Each red–green pair:** what changed — and did I ask for it? Line 18: yes. Line 8: **no.**

Read diffs **in color** — in VS Code, in your agent, on GitHub. Nobody reads the raw text. Scan in this order, every time; slow down only where the answer to ③ is **no**.

One of these is wrong

Last commit — `report.txt`

```
positions: 4
total: 4338.20 CHF
fees: 5 CHF
```

After the assistant's edit

```
positions: 4
total: 4338.20 CHF
fees: 10 CHF
```

- UBSG is 40 × 25.00 = **exactly 1000.00**. With `>`, UBSG now pays 5.
- The rounding you asked for changed nothing you can see. The change you did not ask for moved the fees. No error anywhere: **failure kind 3 — it runs and lies.**
- The assistant's "Done" is a claim. **The diff is the evidence.**

TERMINAL

Say no: `git restore`

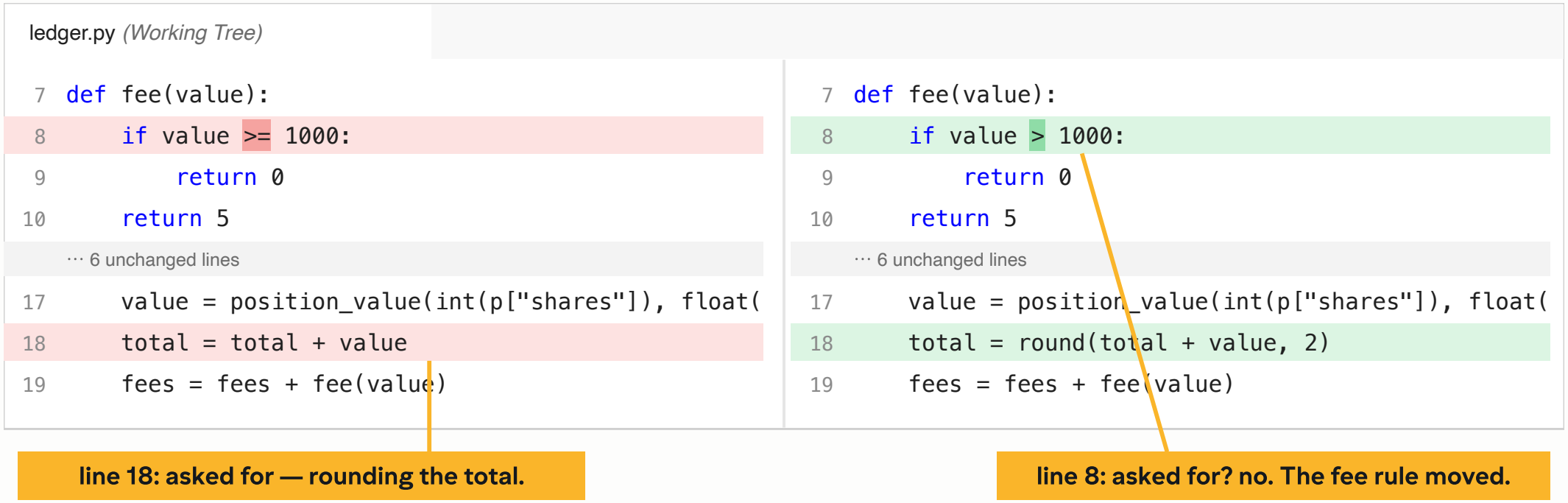
```
$ git restore ledger.py
$ git status --short        prints nothing: clean again
$ python ledger.py
wrote report.txt
$ cat report.txt
positions: 4
total: 4338.20 CHF
fees: 5 CHF                 the rule is back
```

`git restore ledger.py` puts back the version from the last commit. `git status --short` prints nothing: the folder matches the snapshot again. Fees are 5 CHF, as the rule says.

Then ask again — and this time say "change nothing else".

VS CODE

The same check, in VS Code



Red = the last commit. Green = the file in your folder now. Click the file in Source Control to open this view.

Click the file in Source Control to see this view. To throw the edit away: **Discard Changes** (the curved-arrow button) on the file — that is `git restore`.

⚠ WARNING

The rule for working with assistants

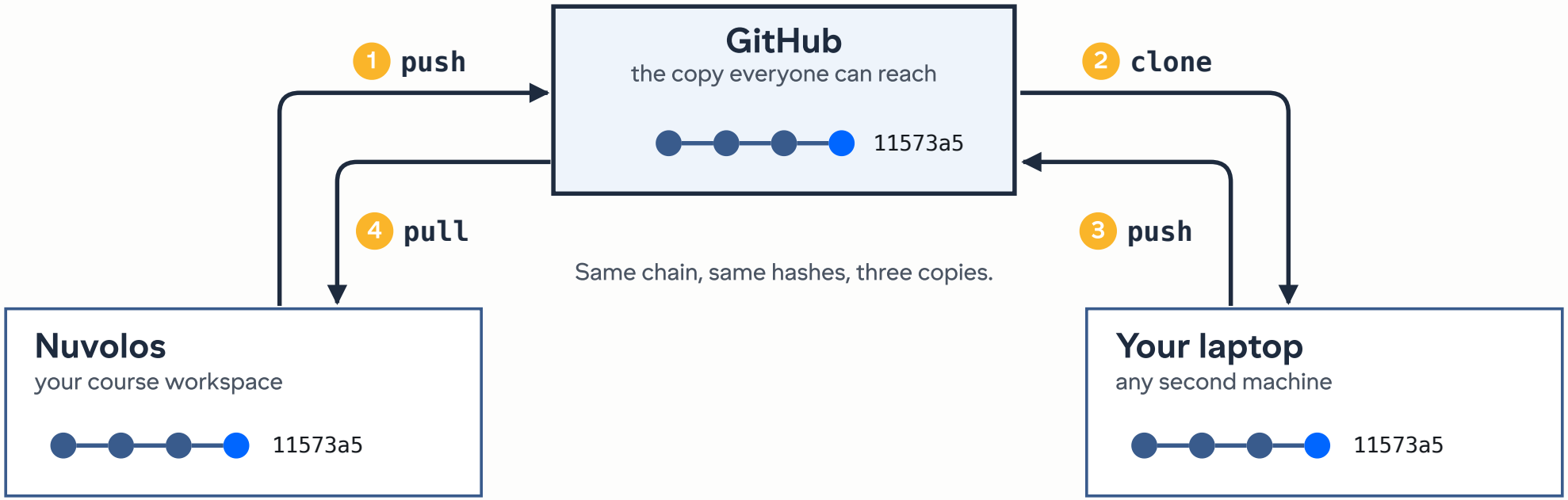
> **This is not a made-up risk.** In 2026, coding agents still change things nobody asked them to change: a threshold, a default, a test they "fixed" so it passes. Not every time — often enough that professionals check every diff.

1. **Commit first.** Clean working tree before the assistant starts.
2. **Let it work.**
3. `git status` — which files did it touch?
4. `git diff` — scan the red and green. Is each change something you asked for?
5. **Run it.** Check one number by hand.
6. **Keep** (`add`, `commit`) or **throw away** (`restore`).

Without step 1, step 6 has nothing to go back to.

IDEA

One history, many copies



A **remote** is another copy of your repository, somewhere else — usually on GitHub. Its usual name is `origin`.

The three remote commands

Command	What it does	How often
`git clone <address>`	copy a repository from GitHub to this machine	**once** per machine
`git push`	send your new commits to GitHub	when you finish working
`git pull`	bring new commits from GitHub to here	when you start working

To use GitHub you need a free account. This course does not require one — but you will meet GitHub in almost every project you join.

TERMINAL

Connect a remote and push

```
$ git remote add origin ../github/ledger.git
$ git remote -v
origin  ../github/ledger.git (fetch)
origin  ../github/ledger.git (push)
$ git push -u origin main
To ../github/ledger.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

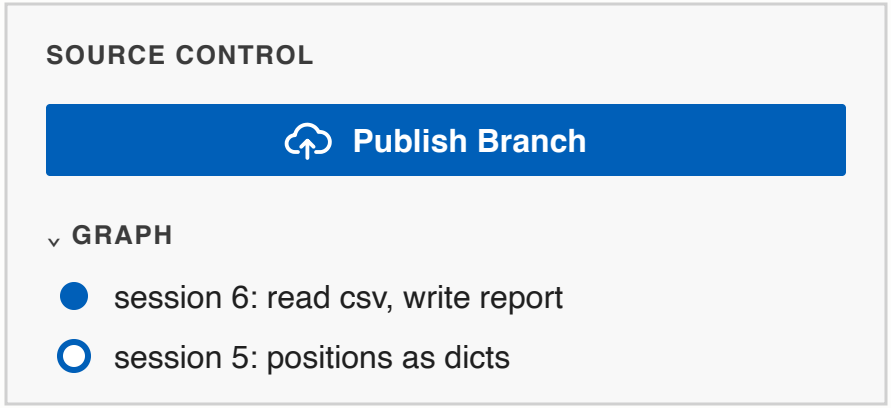
the remote's usual name

In this demo a folder stands in for GitHub. On GitHub the address is `https://github.com/<you>/ledger.git`. `-u` = remember this pairing: after the first time, `git push` alone is enough.

VS CODE

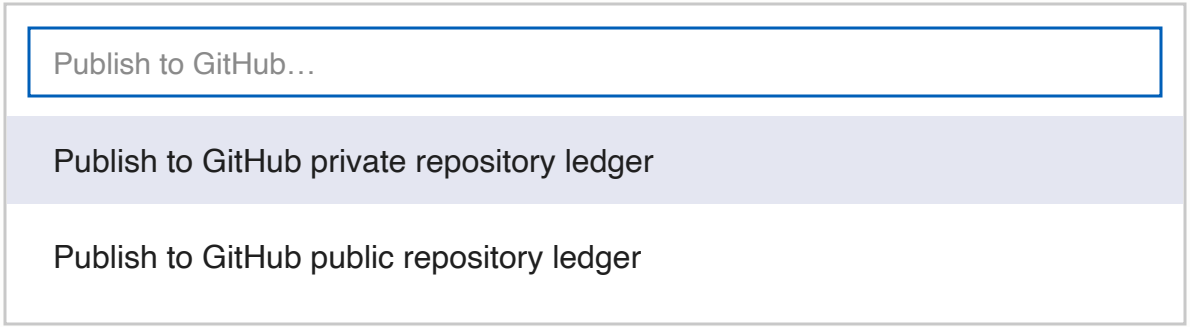
The first push, in VS Code

① No remote yet: the button says Publish



`Publish = git remote add + git push -u`

② VS Code asks where to put it



The first time, VS Code asks you to sign in to GitHub in the browser. Private: only you can see it.

VS Code also creates the repository on GitHub for you.

TERMINAL

Clone it on a second machine

```
$ git clone github/ledger.git laptop
Cloning into 'laptop'...
done.
$ git log --oneline -1
cd81ebf session 6: read csv, write report
```

the same hash as on Nuvolos

`clone` = a full copy: every file **and** every commit. The newest commit on the laptop is `cd81ebf` — **the same hash** as on Nuvolos. Same contents, same name.

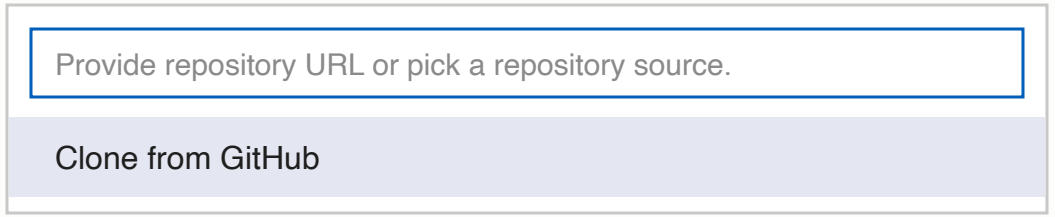
VS CODE

Clone, in VS Code

① Ctrl+Shift+P (Cmd+Shift+P on a Mac), type "clone"



② Paste the address from GitHub




```
Git: Clone = git clone https://github.com/<you>/ledger.git — then choose the folder it goes in
```

Clone once per machine. After that, you pull.

TERMINAL

Work on the laptop, push

```
$ git add README.md
$ git commit -m "add a README"
[main 11573a5] add a README
 1 file changed, 3 insertions(+)
 create mode 100644 README.md
$ git push
To /files/github/ledger.git
   cd81ebf..11573a5  main -> main
```

made on the laptop

from this commit to that one

The laptop made a new commit, `11573a5`, and sent it to GitHub. `cd81ebf..11573a5` reads "from this commit to that one".

Nuvolos does not have it yet.

TERMINAL

Back on Nuvolos: what is new?

```
$ git fetch
From ../github/ledger
   cd81ebf..11573a5  main       -> origin/main
$ git status
On branch main
Your branch is behind 'origin/main' by 1 commit, and can be fast-forwarded.
  (use "git pull" to update your local branch)
nothing to commit, working tree clean
```

the next command

`git fetch` only **looks**: it downloads the news, changes no file. Now `git status` knows GitHub is one commit ahead — and tells you the next command.

TERMINAL

`git pull`

```
$ git pull
Updating cd81ebf..11573a5
Fast-forward
 README.md | 3 +++
 1 file changed, 3 insertions(+)
 create mode 100644 README.md
$ git log --oneline
11573a5 add a README
cd81ebf session 6: read csv, write report
3c6342e session 5: positions as dicts
003891f session 4: functions
9de8ebc session 3: loop and total
ee7054a session 2: value one position
916acb4 session 1: first run
```

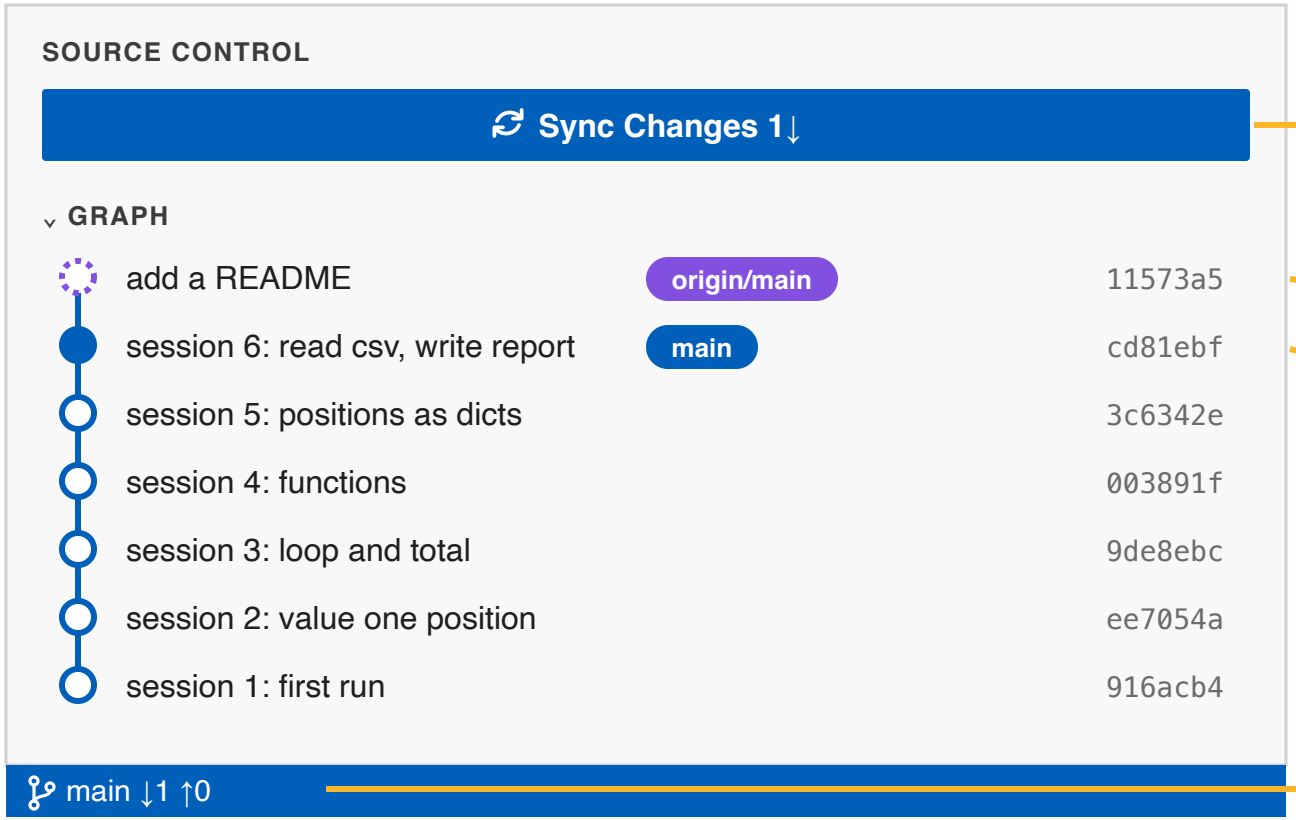
main simply moves ahead

arrived

`README.md` arrived. Nuvolos, GitHub and the laptop now hold the same chain. `Fast-forward` = your `main` simply moved ahead; nothing to mix.

VS CODE

Push and pull, in VS Code



`Sync = git pull, then git push`

`GitHub has one commit you don't`

`your folder stands here`

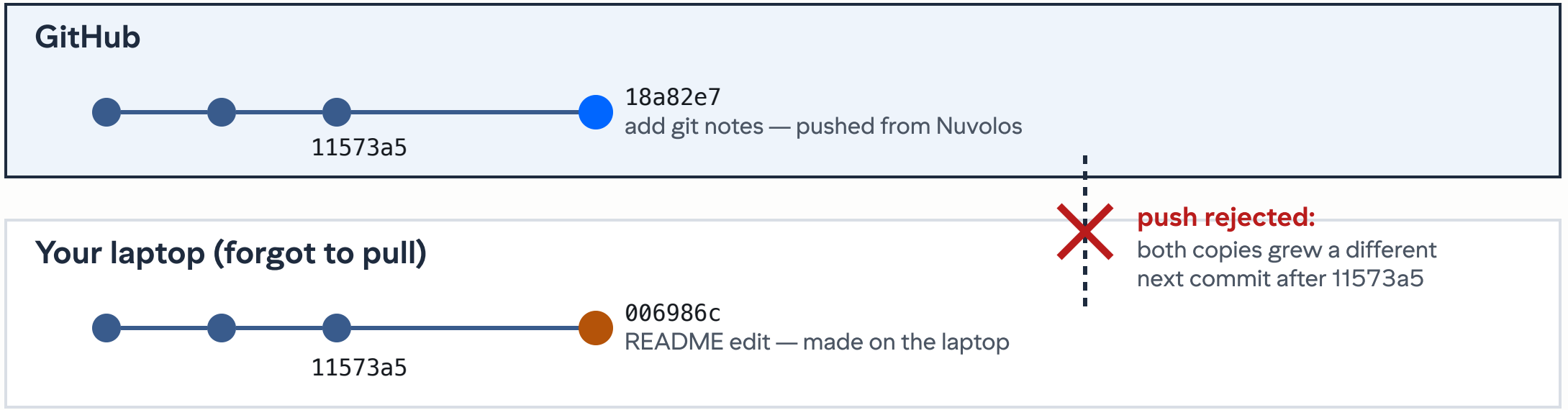
No remote yet? The same button reads **Publish Branch** — that is the first push.

To copy a repository to a new machine: Command Palette → **Git: Clone**.

`↓1: one to pull · ↑0: none to push`

IDEA

What goes wrong: two copies grow apart



Nuvolos pushed a commit. The laptop, without pulling first, made a different commit on top of the same parent. Now there are two different "next" commits.

TERMINAL

The rejected push

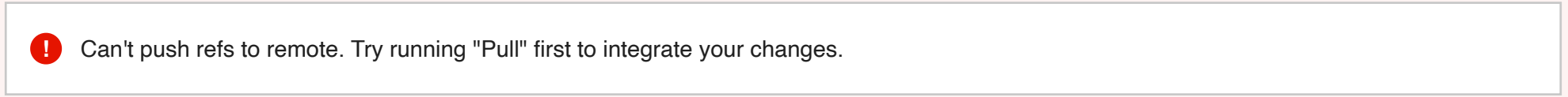
```
$ git commit -m "README: say where the output goes"
[main 006986c] README: say where the output goes
 1 file changed, 2 insertions(+)
$ git push
To /files/github/ledger.git
 ! [rejected]        main -> main (fetch first)   refused
error: failed to push some refs to '/files/github/ledger.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.   what git suggests
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
```

Git refuses to throw away the commit on GitHub. **Nothing was lost.** It is protecting you.

⚠ WARNING

The rule that keeps you out of it

Push, when GitHub has a commit you don't




```
terminal: ! [rejected] main -> main (fetch first)
```

Pull when you start. Push when you finish.

With one person and two machines, that rule is enough — the two copies never grow apart.

If a push is rejected anyway:

- **Do not use `--force`.** It makes GitHub drop the commit it has. (Some assistants suggest it. Say no.)
- Joining two different histories is called a **merge**. It is not part of this course. Ask a TA — it takes two minutes with help.

Common messages, and what they mean

You see	It means	Do
`fatal: not a git repository (or any of the parent directories): .git`	you are not inside a repository	`cd` into your project folder
`*** Please tell me who you are.`	new machine, no identity	the two `git config` lines
`nothing to commit, working tree clean`	no change since the last commit	nothing — all saved
`nothing added to commit but untracked files present`	you forgot `add`	`git add <file>`
`! [rejected] … (fetch first)`	GitHub has a commit you don't	`git pull` first; ask if it fails

The dictionary: every button is a command

You want to	Terminal	VS Code
start a history	`git init`	Initialize Repository
see what changed	`git status`	the Changes list
see the change itself	`git diff`	click the file
put a change in the next commit	`git add`	+
take it back out	`git restore --staged`	−
save a snapshot	`git commit -m "…"`	✓ Commit
throw an edit away	`git restore`	Discard Changes
read the history	`git log`	Graph
send / fetch commits	`git push` / `git pull`	Sync Changes
copy a repository	`git clone`	Git: Clone

Git: what you can now do

- Say what git keeps (snapshots), where (`.git`), and how each one is named (a hash).
- Run the everyday loop: `status` → `diff` → `add` → `commit`.
- Choose what goes into a commit, and keep outputs and secrets out with `.gitignore`.
- Read the history, and one commit's diff.
- Check an assistant's work with `git diff`, and throw it away with `git restore`.
- Clone, pull, push — and not panic at a rejected push.

Reminder: none of this is on the midterm. Part 2 is.

PART 2

Review: sessions 1–6

Everything the midterm asks, once more, on the scripts you already know. Less depth, all the ground.

The midterm: how it works

- **Monday 2 November**, in class, on **Moodle**, on your own laptop. **60 minutes**, proctored.
- You **read** code. Nothing is run during the test.
- Questions go from **easy to hard**. Harder questions are worth more. Move freely between them.
- Answer forms: **choose one answer** · **choose from a list inside the code** · **match items** · **type a number**.
- A question with several parts gives points for each part you get right.
- Your neighbour may see **another version** of the same question.
- A few **bonus** questions come at the end (comprehensions, `lambda`). Their points come **on top**.
- Random guessing is corrected for. The exact scale is written on the test.

Not on the midterm: git · reading tracebacks · writing classes · slides marked **extra**.

How to use this part

Each slide pins **a script from its own session** on the left. For every one: **cover the output, say what it prints, then check.** That is the midterm skill.

The midterm asks	Train it by
What does this print?	tracing on paper
Which line does not follow the brief?	reading every line you are shown
Which kind of failure is this?	won't start · crashes · **runs and lies**
Is this output plausible? Why not?	checking one number by hand
What goes in the gap?	trying each choice in your head
What does this function do?	saying it in one English sentence
Questions about a longer program	reading it once, top to bottom, first
Which tool fits the task?	knowing what each tool is for

The practice items at the end of this part have the same forms. They are **not** the midterm questions.

REMINDER

The machine underneath (session 1)

A variable remembers the result, not the recipe. After `value = shares * price`, changing `price` does not change `value`.

Paths. An absolute path starts at the root `/` and means the same from anywhere. A relative path starts in the **working directory**, the folder the program is "in":

Working directory	Path in the code	File Python opens
`/home/you/course`	`"notes.txt"`	`/home/you/course/notes.txt`
`/home/you/course`	`"week1/notes.txt"`	`/home/you/course/week1/notes.txt`
`/home/you/course`	`"../notes.txt"`	`/home/you/notes.txt`

Three ways code fails: it **won't start** (Python cannot read it) · it **crashes** (it starts, then stops with a message) · it **runs and lies** (no error, wrong result).

position.py

```python
# position.py — one position of the ledger
ticker = "NESN"
raw = "4"                # text, as read from a file
shares = 10              # how many we hold
price = 102.3
cash = 1000.0
last_sale = None
print(raw * 2, shares * 2, type(raw), type(shares))

shares = shares + 6   # buy 6 more
cash -= 6 * price
value = shares * price
weight = value / (value + cash)
can_buy = int(cash // price)
rest = cash % price
in_2y = value * 1.05 ** 2

valid = len(ticker) <= 5 and shares > 0
alert = weight > 0.5 or not valid

print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
print(f"cash {cash:.2f}, weight {weight:.2f}")
print(can_buy, rest, round(in_2y, 2))
print(valid, alert, last_sale)
```

text, as read from a file

same *, two answers

```
$ python position.py
44 8 <class 'str'> <class 'int'>
386.20000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.30000000000005 1127.86
True True None
```

REMINDER

Every value has a type (session 2)

`str` text · `int` whole numbers · `float` decimals · `bool` `True` / `False` · `None` no value yet.

The type decides what an operator does: `"4" * 2` repeats the text, `4 * 2` multiplies.

`type(x)` tells you. **`None` is not `0`.**

position.py

```python
# position.py — one position of the ledger
ticker = "NESN"
raw = "4"              # text, as read from a file
shares = int(raw)
price = 102.3
cash = 1000.0
last_sale = None
print(raw * 2, shares * 2, type(raw), type(shares))

shares = shares + 6    # buy 6 more
cash -= 6 * price
value = shares * price
weight = value / (value + cash)
can_buy = int(cash // price)
rest = cash % price
in_2y = value * 1.05 ** 2

valid = len(ticker) <= 5 and shares > 0
alert = weight > 0.5 or not valid

print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
print(f"cash {cash:.2f}, weight {weight:.2f}")
print(can_buy, rest, round(in_2y, 2))
print(valid, alert, last_sale)
```

right side first

```
$ python position.py
44 8 <class 'str'> <class 'int'>
386.20000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.30000000000005 1127.86
True True None
```

REMINDER

`=` is state: trace it (session 2)

After line	`shares`	`cash`
4	`4`	—
6	`4`	`1000.0`
10	`10`	`1000.0`
11	`10`	`386.20000000000005`

`=` computes the right side **now**, then stores it. Tracing by hand is the main midterm skill.

position.py

```python
# position.py — one position of the ledger
ticker = "NESN"
raw = "4"              # text, as read from a file
shares = int(raw)
price = 102.3
cash = 1000.0
last_sale = None
print(raw * 2, shares * 2, type(raw), type(shares))

shares = shares + 6   # buy 6 more
cash -= 6 * price
value = shares * price
weight = value / (value + cash)
can_buy = int(cash // price)
rest = cash % price
in_2y = value * 1.05 ** 2

valid = len(ticker) <= 5 and shares > 0
alert = weight > 0.5 or not valid

print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
print(f"cash {cash:.2f}, weight {weight:.2f}")
print(can_buy, rest, round(in_2y, 2))
print(valid, alert, last_sale)
```

floor division

```
$ python position.py
44 8 <class 'str'> <class 'int'>
386.20000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.30000000000005 1127.86
True True None
```

never == on money

REMINDER

Arithmetic, and floats (session 2)

`/` always gives a float · `//` floor division · `%` remainder · `**` power. **When in doubt, add parentheses.**

A float stores the closest number it can. So:

- **Never** compare floats with `==`. Use a tolerance (line 21).
- **Round only for display**, at the very end.

position.py **CHANGED ON THIS SLIDE**

```python
# position.py — one position of the ledger
ticker = "NESN"
raw = "4"              # text, as read from a file
shares = int(raw)
price = 102.3
cash = 1000.0
last_sale = None
print(raw * 2, shares * 2, type(raw), type(shares))

shares = raw + 6   # buy 6 more
cash -= 6 * price
value = shares * price
weight = value / (value + cash)
can_buy = int(cash // price)
rest = cash % price
in_2y = value * 1.05 ** 2

valid = len(ticker) <= 5 and shares > 0
alert = weight > 0.5 or not valid

print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
print(f"cash {cash:.2f}, weight {weight:.2f}")
print(can_buy, rest, round(in_2y, 2))
print(valid, alert, last_sale)
```

text + number

```
$ python position.py
44 8 <class 'str'> <class 'int'>
Traceback (most recent call last):
  File "position.py", line 10, in <module>
    shares = raw + 6   # buy 6 more
             ~~~~^~~
TypeError: can only concatenate str (not "int") to str
```

REMINDER

Convert first, then compute (session 2)

Numbers coerce (`int` → `float`) automatically. **Text never does.**

`raw` is the text `"4"`. Line 10 adds a number to it: Python refuses, with a `TypeError`. Kind 2: it crashes.

`int(raw)` converts. `int("4.5")` fails too: a `ValueError`.

position.py **CHANGED ON THIS SLIDE**

```python
# position.py — one position of the ledger
ticker = "NESN"
raw = "4"              # text, as read from a file
shares = int(raw)
price = 102.3
cash = 1000.0
last_sale = None
print(raw * 2, shares * 2, type(raw), type(shares))

shares == shares + 6   # buy 6 more
cash -= 6 * price
value = shares * price
weight = value / (value + cash)
can_buy = int(cash // price)
rest = cash % price
in_2y = value * 1.05 ** 2

valid = len(ticker) <= 5 and shares > 0
alert = weight > 0.5 or not valid

print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
print(f"cash {cash:.2f}, weight {weight:.2f}")
print(can_buy, rest, round(in_2y, 2))
print(valid, alert, last_sale)
```

asks; stores nothing

```
$ python position.py
44 8 <class 'str'> <class 'int'>
386.20000000000005 False True
NESN: 4 shares, 409.20 CHF
cash 386.20, weight 0.51
3 79.30000000000005 451.14
True True None
```

REMINDER

`=` stores, `==` asks (session 2)

One character, a silent lie: line 10 now **compares** and throws the answer away. No error. The report shows 4 shares, not 10. Kind 3: **it runs and lies**.

`and`, `or`, `not` combine comparisons (lines 18–19). Order: comparisons, then `not`, then `and`, then `or`.

position.py

```python
# position.py — one position of the ledger
ticker = "NESN"
raw = "4"              # text, as read from a file
shares = int(raw)
price = 102.3
cash = 1000.0
last_sale = None
print(raw * 2, shares * 2, type(raw), type(shares))

shares = shares + 6   # buy 6 more
cash -= 6 * price
value = shares * price
weight = value / (value + cash)
can_buy = int(cash // price)
rest = cash % price
in_2y = value * 1.05 ** 2

valid = len(ticker) <= 5 and shares > 0
alert = weight > 0.5 or not valid

print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
print(f"cash {cash:.2f}, weight {we
print(can_buy, rest, round(in_2y, 2))
print(valid, alert, last_sale)
```

display only

```
$ python position.py
44 8 <class 'str'> <class 'int'>
386.20000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.30000000000005 1127.86
True True None
```

REMINDER

What you see is not what is stored (session 2)

An f-string puts values inside text. `:,.2f` = thousands separator, two decimals.

Formatting changes the display. It does not change the value. `value` still holds `1023.0`.

ledger.py

```python
# ledger.py — the whole ledger: lists, a dict, loops
tickers = ["AAPL", "NESN", "VOO"]
shares = [10, 4, 2]
price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
print(tickers[0], tickers[-1], tickers[1:], price["NESN"])

total = 0.0
for i in range(len(tickers)):
    value = shares[i] * price[tickers[i]]
    total += value
    if value >= 1500:
        size = "large"
    elif value >= 500:
        size = "medium"
    else:
        size = "small"
    print(f"{tickers[i]:<5} {value:>9,.2f}  {size}")

years = 0
grown = total
while grown < 2 * total:
    grown *= 1.05
    years += 1
print(f"total {total:,.2f}, doubles in {years} years")
```

the last

```
$ python ledger.py
AAPL VOO ['NESN', 'VOO'] 102.3
AAPL   1,905.00  large
NESN     409.20  small
VOO    1,024.00  medium
total 3,338.20, doubles in 15 years
```

stop not included

REMINDER

Lists: positions from 0 (session 3)

`tickers[0]` is the first. `tickers[-1]` the last.
`tickers[3]` is one too far: `IndexError`.

A slice `[1:]` takes a piece. **Positions from 0, stop not included.** Slices never crash.

Lists can change (`.append`, `.sort`). Strings cannot.
`sorted(x)` gives back a **new** list. `.sort()` and `.append()` change the list itself and give back **`None`**.

ledger.py

```python
# ledger.py — the whole ledger: lists, a dict, loops
tickers = ["AAPL", "NESN", "VOO"]
shares = [10, 4, 2]
price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
print(tickers[0], tickers[-1], tickers[1:], price["NESN"])

total = 0.0
for i in range(len(tickers)):
    value = shares[i] * price[tickers[i]]
    total += value
    if value >= 1500:
        size = "large"
    elif value >= 500:
        size = "medium"
    else:
        size = "small"
    print(f"{tickers[i]:<5} {value:>9,.2f}  {size}")

years = 0
grown = total
while grown < 2 * total:
    grown *= 1.05
    years += 1
print(f"total {total:,.2f}, doubles in {years} years")
```

1. before the loop

2. inside

```
$ python ledger.py
AAPL VOO ['NESN', 'VOO'] 102.3
AAPL   1,905.00  large
NESN     409.20  small
VOO    1,024.00  medium
total 3,338.20, doubles in 15 years
```

REMINDER

The loop and the accumulator (session 3)

pass	ticker	`total` after line 10
1	AAPL	1905.0
2	NESN	2314.2
3	VOO	3338.2

Start **before** the loop, add **inside**, use **after**. Indentation decides what is inside.

ledger.py

```python
# ledger.py — the whole ledger: lists, a dict, loops
tickers = ["AAPL", "NESN", "VOO"]
shares = [10, 4, 2]
price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
print(tickers[0], tickers[-1], tickers[1:], price["NESN"])

total = 0.0
for i in range(len(tickers)):
    value = shares[i] * price[tickers[i]]
    total += value
    if value >= 1500:
        size = "large"
    elif value >= 500:
        size = "medium"
    else:
        size = "small"
    print(f"{tickers[i]:<5} {value:>9,.2f}  {size}")

years = 0
grown = total
while grown < 2 * total:
    grown *= 1.05
    years += 1
print(f"total {total:,.2f}, doubles in {years} years")
```

biggest first

```
$ python ledger.py
AAPL VOO ['NESN', 'VOO'] 102.3
AAPL   1,905.00  large
NESN     409.20  small
VOO    1,024.00  medium
total 3,338.20, doubles in 15 years
```

REMINDER

`if`, `elif`, `else`: the first `True` wins (session 3)

Exactly one block of the chain runs: the first whose test is `True`.

- In a chain of thresholds, test the **biggest** first.
- `>=` or `>`? The two differ only **at** the boundary. **The brief decides**, not the code.

ledger.py

```python
# ledger.py — the whole ledger: lists, a dict, loops
tickers = ["AAPL", "NESN", "VOO"]
shares = [10, 4, 2]
price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
print(tickers[0], tickers[-1], tickers[1:], price["NESN"])

total = 0.0
for i in range(len(tickers)):
    value = shares[i] * price[tickers[i]]
    total += value
    if value >= 1500:
        size = "large"
    elif value >= 500:
        size = "medium"
    else:
        size = "small"
    print(f"{tickers[i]:<5} {value:>9,.2f}  {size}")

years = 0
grown = total
while grown < 2 * total:
    grown *= 1.05
    years += 1
print(f"total {total:,.2f}, doubles in {years} years")
```

read inside out

```
$ python ledger.py
AAPL VOO ['NESN', 'VOO'] 102.3
AAPL   1,905.00  large
NESN     409.20  small
VOO    1,024.00  medium
total 3,338.20, doubles in 15 years
```

REMINDER

Dicts: look up by key (session 3)

A dict maps **keys** to **values**: `price["NESN"]` is `102.3`.

- A list complains about a **position**: `IndexError`. A dict complains about a **key**: `KeyError: 'MSFT'`.
- Ask first: `"MSFT" in price`, or `price.get("MSFT", 0.0)`.
- A list of dicts is a table: a row is a dict.

ledger.py

```python
# ledger.py — the whole ledger: lists, a dict, loops
tickers = ["AAPL", "NESN", "VOO"]
shares = [10, 4, 2]
price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
print(tickers[0], tickers[-1], tickers[1:], price["NESN"])

total = 0.0
for i in range(len(tickers)):
    value = shares[i] * price[tickers[i]]
    total += value
    if value >= 1500:
        size = "large"
    elif value >= 500:
        size = "medium"
    else:
        size = "small"
    print(f"{tickers[i]:<5} {value:>9,.2f}  {size}")

years = 0
grown = total
while grown < 2 * total:
    grown *= 1.05
    years += 1
print(f"total {total:,.2f}, doubles in {years} years")
```

checked before each pass

```
$ python ledger.py
AAPL VOO ['NESN', 'VOO'] 102.3
AAPL   1,905.00  large
NESN     409.20  small
VOO    1,024.00  medium
total 3,338.20, doubles in 15 years
```

REMINDER

`while`: repeat while it is true (session 3)

`for` goes through the items of a collection. `while` repeats **as long as** a test is `True`, when you do not know how many passes you need.

If the test never becomes `False`, the loop never stops.

Two names, one list (session 3)

```
shares = [10, 4, 2]
backup = shares
backup[1] = 6
print(shares)
```

```
[10, 6, 2]
```

`backup = shares` does **not** copy the list. It puts a second name on the **same** list. The fix: `backup = shares.copy()`.

portfolio.py

```python
# portfolio.py — the ledger, organised into functions
import math

def position_value(shares, price):
    """Value of one position, in CHF."""
    return shares * price

def total_value(tickers, shares, price):
    total = 0.0
    for i in range(len(tickers)):
        total += position_value(shares[i], price[tickers[i]])
    return total

def years_to_double(rate=0.05):
    return math.ceil(math.log(2) / math.log(1 + rate))

tickers = ["AAPL", "NESN", "VOO"]
shares = [10, 4, 2]
price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}

total = total_value(tickers, shares, price)
print(f"total {total:,.2f}", round(total, 1))
print(years_to_double(), year
print(position_value(price=102.3, shares=4), sum(shares))
print(", ".join(tickers).lower(), type(total_value))
```

→ round(3338.2, 1) → 3338.2

```
$ python portfolio.py
total 3,338.20 3338.2
15 11
409.2 16
aapl, nesn, voo <class 'function'>
```

REMINDER

A call is replaced by its value (session 4)

Python runs the call, then puts its **return value** where the call was. Calls inside calls: read from the inside out.

A name is not a call: `total_value` without `(...)` is the function itself.

portfolio.py **CHANGED ON THIS SLIDE**

```python
# portfolio.py — the ledger, organised into functions
import math

def position_value(shares, price):
    """Value of one position, in CHF."""
    print(shares * price)

def total_value(tickers, shares, price):
    total = 0.0
    for i in range(len(tickers)):
        total += position_value(shares[i], price[tickers[i]])
    return total

def years_to_double(rate=0.05):
    return math.ceil(math.log(2) / math.log(1 + rate))

tickers = ["AAPL", "NESN", "VOO"]
shares = [10, 4, 2]
price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}

total = total_value(tickers, shares, price)
print(f"total {total:,.2f}", round(total, 1))
print(years_to_double(), years_to_double(rate=0.07))
print(position_value(price=102.3, shares=4), sum(shares))
print(", ".join(tickers).lower(), type(total_value))
```

shows it, returns None

```
$ python portfolio.py
1905.0
Traceback (most recent call last):
  File "portfolio.py", line 21, in <module>
    total = total_value(tickers, shares, price)
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "portfolio.py", line 11, in total_value
    total += position_value(shares[i], price[tickers[i]])
TypeError: unsupported operand type(s) for +=: 'float' and 'NoneType'
```

REMINDER

`print` is not `return` (session 4)

A value on the screen is not a value in a variable. `print` shows `1905.0`, but the call returns `None`, and line 11 cannot add `None`.

Every function returns something. Without `return`, it returns `None`.

portfolio.py

```python
# portfolio.py — the ledger, organised into functions
import math

def position_value(shares, price):
    """Value of one position, in CHF."""
    return shares * price

def total_value(tickers, shares, price):
    total = 0.0
    for i in range(len(tickers)):
        total += position_value(shares[i], price[tickers[i]])
    return total

def years_to_double(rate=0.05):
    return math.ceil(math.log(2) / math.log(1 + rate))

tickers = ["AAPL", "NESN", "VOO"]
shares = [10, 4, 2]
price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}

total = total_value(tickers, shares, price)
print(f"total {total:,.2f}", round(total, 1))
print(years_to_double(), years_to_double(rate=0.07))
print(position_value(price=102.3, shares=
print(", ".join(ticke(total_value))
```

a default

a keyword argument

by name: any order

```
$ python portfolio.py
total 3,338.20 3338.2
15 11
409.2 16
aapl, nesn, voo <class 'function'>
```

REMINDER

Signatures and arguments (session 4)

Positional arguments are matched by order, **keyword** arguments by name. A **default** is used when you give nothing.

Rule: **all positional arguments first, then keywords.** Too few arguments: `TypeError … missing 1 required positional argument`.

portfolio.py **CHANGED ON THIS SLIDE**

```python
# portfolio.py — the ledger, organised into functions
import math

def position_value(shares, price):
    """Value of one position, in CHF."""
    return shares * price

def total_value(tickers, shares, price):
    total = 0.0
    for i in range(len(tickers)):
        total += position_value(shares[i], price[tickers[i]])
    return total

def years_to_double(rate=0.05):
    return math.ceil(math.log(2) / math.log(1 + rate))

tickers = ["AAPL", "NESN", "VOO"]
shares = [10, 4, 2]
price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}

total = total_value(tickers, shares, price)
print(f"total {total:,.2f}", round(total, 1))
print(years_to_double(), years_to_double(rate=0.07))
print(position_value(price=102.3, shares=4), sum(shares))
print(i)   # lived only inside the call
```

```
$ python portfolio.py
total 3,338.20 3338.2
15 11
409.2 16
Traceback (most recent call last):
  File "portfolio.py", line 25, in <module>
    print(i)
          ^
NameError: name 'i' is not defined. Did you mean: 'id'?
```

↺ REMINDER

Scope: a function's names are its own (session 4)

`i` exists inside `total_value`, during the call. After the call, it is gone: `NameError`.

The `total` inside the function and the `total` outside are two different names.

To get something out of a function, return it.

portfolio.py

```python
# portfolio.py — the ledger, organised into functions
import math

def position_value(shares, price):
    """Value of one position, in CHF."""
    return shares * price

def total_value(tickers, shares, price):
    total = 0.0
    for i in range(len(tickers)):
        total += position_value(shares[i], price[tickers[i]])
    return total

def years_to_double(rate=0.05):
    return math.ceil(math.log(2) / math.log(1 + rate))

tickers = ["AAPL", "NESN", "VOO"]
shares = [10, 4, 2]
price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}

total = total_value(tickers, shares, price)
print(f"total {total:,.2f}", round(total, 1))
print(years_to_double(), years_to_double(rate=0.07))
print(position_value(price=102.3, shares=4), sum(shares))
print(", ".join(tickers).lower(), type(total_value))
```

ceil, from math

left to right

```
$ python portfolio.py
total 3,338.20 3338.2
15 11
409.2 16
aapl, nesn, voo <class 'function'>
```

REMINDER

Methods and modules: the dot (session 4)

`math.ceil` is the `ceil` that lives in the module `math`. `tickers` … `.lower()` are methods: tools that belong to a type.

- `import math` · `from math import ceil` · `import numpy as np`
- **The class decides the tools:** `sum(shares)` on a list, `arr.sum()` on an array.

prices.py

```python
# prices.py — nested data, read in one line
ledger = {
    "AAPL": {"shares": 10, "prices": [185.0, 188.2, 190.5]},
    "NESN": {"shares": 4, "prices": [104.1, 103.0, 102.3]},
    "VOO": {"shares": 2, "prices": [505.0, 509.9, 512.0]},
}
print(ledger["NESN"]["prices"][-1])
print(ledger["NESN"]["prices"][1:])

last = {t: d["prices"][-1] for t, d in ledger.items()}
values = [d["shares"] * last[t] for t, d in ledger.items()]
rising = [t for t, d in ledger.items()
          if d["prices"][-1] > d["prices"][0]]
print(last)
print(values, sum(values))
print(rising)

by_value = sorted(ledger, reverse=True,
                  key=lambda t: ledger[t]["shares"] * last[t])
print(by_value)
change = lambda p: (p[-1] - p[0]) / p[0]
print({t: round(change(d["prices"]), 4)
       for t, d in ledger.items()})
```

dict → dict → list → float

```
$ python prices.py
102.3
[188.2, 190.5]
{'AAPL': 190.5, 'NESN': 102.3, 'VOO': 512.0}
[1905.0, 409.2, 1024.0] 3338.2
['AAPL', 'VOO']
['AAPL', 'VOO', 'NESN']
{'AAPL': 0.0297, 'NESN': -0.0173, 'VOO': 0.0139}
```

REMINDER

Nested data: one look-up at a time (session 5)

Read the chain **left to right**, and ask at each step: **what type do I have now?**

Level	Wrong	Error
dict	a missing key	`KeyError`
list	a position too far	`IndexError`
wrong type	`ledger["NESN"][0]`	`KeyError: 0`

prices.py

```python
# prices.py — nested data, read in one line
ledger = {
    "AAPL": {"shares": 10, "prices": [185.0, 188.2, 190.5]},
    "NESN": {"shares": 4, "prices": [104.1, 103.0, 102.3]},
    "VOO": {"shares": 2, "prices": [505.0, 509.9, 512.0]},
}
print(ledger["NESN"]["prices"][-1])
print(ledger["AAPL"]["prices"][1:])

last = {t: d["prices"][-1] for t, d in ledger.items()}
values = [d["shares"] * last[t] for t, d in ledger.items()]
rising = [t for t, d in ledger.i
          if d["prices"][-1] > d["prices"][0]]
print(last)
print(values, sum(values))
print(rising)

by_value = sorted(ledger, reverse=True,
                  key=lambda t: ledger[t]["shares"] * last[t])
print(by_value)
change = lambda p: (p[-1] - p[0]) / p[0]
print({t: round(change(d["prices"]), 4)
       for t, d in ledger.items()})
```

read from for

a small function, passed in

```
$ python prices.py
102.3
[188.2, 190.5]
{'AAPL': 190.5, 'NESN': 102.3, 'VOO': 512.0}
[1905.0, 409.2, 1024.0] 3338.2
['AAPL', 'VOO']
['AAPL', 'VOO', 'NESN']
{'AAPL': 0.0297, 'NESN': -0.0173, 'VOO': 0.0139}
```

REMINDER

Comprehensions and `lambda` (session 5)

A comprehension is a loop in one line. Read it in three steps: from `for` to the end, then the start, then the brackets. Line 11: "the value of each position, as a list".

A `lambda` is a small function, often passed to another one (`sorted`, `max`).

Say each line in one English sentence. If you cannot, rewrite it as a loop.

On the midterm: **bonus questions only**, at the end.

arrays.py

```python
# arrays.py — numpy: a grid of numbers of one type
import numpy as np

prices = np.array([[185.0, 188.2, 190.5],
                   [104.1, 103.0, 102.3],
                   [505.0, 509.9, 512.0]])
shares = np.array([10, 4, 2])
print(prices.shape, prices.dtype, prices.ndim)
print(prices[1, 2], prices[1], prices[:, -1])
print(prices[:2, 1:])

values = shares * prices[:, -1]
print(values, values.sum())
print(prices.mean(axis=1).round(2), prices.max(axis=0))
history = shares[:
print(history.sum(axis=0))
returns = prices[:, 1:] / prices[:, :-1] - 1
print(returns.round(4))
print(values[values > 1000], (returns > 0).sum())
```

one result per row

```
$ python arrays.py
(3, 3) float64 2
102.3 [104.1 103.  102.3] [190.5 102.3 512. ]
[[188.2 190.5]
 [103.  102.3]]
[1905.   409.2 1024. ] 3338.2
[187.9  103.13 508.97] [505.  509.9 512. ]
[3276.4 3313.8 3338.2]
[[ 0.0173  0.0122]
 [-0.0106 -0.0068]
 [ 0.0097  0.0041]]
[1905. 1024.] 4
```

REMINDER

numpy: arrays, shapes, axes (session 5)

Arithmetic on arrays works **element by element**, with no loop (line 12).

- `shape`: how many elements along each direction.
- `axis=`: which direction to collapse. The wrong axis **runs fine** — and lies.
- Masks: `values[values > 1000]`. Combine with `&`, not `and`.

On the midterm: **a little**, at reading level.

tables.py

```python
# tables.py — pandas: a table with names
import pandas as pd

df = pd.DataFrame({"ticker": ["AAPL", "MSFT", "NESN", "VOO"],
                   "sector": ["tech", "tech", "food", "fund"],
                   "shares": [10, 3, 4, 2],
                   "price": [190.5, 410.0, 102.3, 512.0]})
df["value"] = df["shares"] * df["price"]
print(df)
print(df.shape, df["value"].sum())
print(df.loc[2, "ticker"], df.iloc[0, 3])
big = df[df["value"] > 1000]
print(big[["ticker", "value"]])
df["size"] = df["value"].apply(
    lambda v: "large" if v > 1500 else "small")
print(df.groupby("sector")["value"].sum())
```

by label

split, apply, combine

$ python tables.py

```
  ticker sector  shares  price   value
0   AAPL   tech      10  190.5  1905.0
1   MSFT   tech       3  410.0  1230.0
2   NESN   food       4  102.3   409.2
3    VOO   fund       2  512.0  1024.0
(4, 5) 4568.2
NESN 190.5
  ticker   value
0   AAPL  1905.0
1   MSFT  1230.0
3    VOO  1024.0
sector
food     409.2
fund    1024.0
tech    3135.0
Name: value, dtype: float64
```

REMINDER

pandas: a table with names (session 5)

A **DataFrame** is a table; a column is a **Series**. New column: line 8, element by element.

- `.loc` by **label**, `.iloc` by **position**. Know which one you mean.
- Filter rows with a mask (line 13). `groupby` : split, apply, combine.

On the midterm: **a little**, at reading level. pandas comes back in full after the midterm.

report.py

```python
# report.py — ledger.csv in, report.txt out
from pathlib import Path

def load(path):
    rows = []
    with open(path, encoding="utf-8") as f:
        header = f.readline()
        for line in f:
            parts = line.strip().split(",")
            ticker, name, shares, price = parts
            try:
                rows.append((name, int(shares), float(price)))
            except ValueError:
                print(f"skipped {ticker}: bad number")
    return rows

def write_report(rows, path):
    total = 0.0
    with open(path, "w", encoding="utf-8") as out:
        for name, shares, price in rows:
            out.write(f"{name:<9}{shares * price:>10,.2f}\n")
            total += shares * price
        out.write(f"{'total':<9}{total:>10,.2f}\n")
    return total

rows = load(Path("ledger.csv"))
total = write_report(rows, Path("report.txt"))
assert total > 0, "an empty report?"
print(len(rows), "positions, total", round(total, 2))
print(Path("report.txt").read_text())
```

text → list of text

unpacking

```
$ python report.py
skipped MSFT: bad number
3 positions, total 3338.2
Apple      1,905.00
Nestlé       409.20
Vanguard   1,024.00
total      3,338.20
```

REMINDER

From a line of text to numbers (session 6)

A text file is a sequence of lines. Each line ends with an invisible `\n`: `.strip()` removes it, `.split(",")` cuts the fields.

Everything read from a file is **text**. Convert, then compute (session 2).

Always `with open(..., encoding="utf-8")`.

report.py

```python
# report.py — ledger.csv in, report.txt out
from pathlib import Path

def load(path):
    rows = []
    with open(path, encoding="utf-8") as f:
        header = f.readline()
        for line in f:
            parts = line.strip().split(",")
            ticker, name, shares, price = parts
            try:
                rows.append((name, int(shares), float(price)))
            except ValueError:
                print(f"skipped {ticker}: bad number")
    return rows

def write_report(rows, path):
    total = 0.0
    with open(path, "w", encoding="utf-8") as out:
        for name, shares, price in rows:
            out.write(f"{name:<9}{shares * price:>10,.2f}\n")
            total += shares * price
        out.write(f"{'total':<9}{total:>10,.2f}\n")
    return total

rows = load(Path("ledger.csv"))
total = write_report(rows, Path("report.txt"))
assert total > 0, "an empty report?"
print(len(rows), "positions, total", round(total, 2))
print(Path("report.txt").read_text())
```

name the error you expect

erases first

```
$ python report.py
skipped MSFT: bad number
3 positions, total 3338.2
Apple      1,905.00
Nestlé       409.20
Vanguard   1,024.00
total      3,338.20
```

REMINDER

Errors as tools (session 6)

Situation	Do
a problem you did not expect	let it **crash**
bad input you expect, one row harmless	`try` / `except` : **skip**, and say so
bad input that makes the result meaningless	**raise**
must be true if the code is right	**assert** it (line 28)

Never: catch everything and continue quietly.

REMINDER

The bugs we have met (sessions 1–6)

Bug	Seen in	Catch it by
text where a number was meant: `"4" * 2`	session 2	`type()`, `repr()`
float rounding, `==` on money	session 2	a tolerance
boundaries: `>` vs `>=`, off by one	session 3	test the edge case
two names, one list	sessions 3–4	`.copy()`, read the code
wrong axis, wrong broadcasting	session 5	shapes, one number by hand
missing data made up or hidden	session 6	say it, `raise`, `assert`

Almost none of them crash.

Item 1 · What does this print?

```
shares = [10, 4, 2]
total = 0
for n in shares:
    if n > 3:
        total += n
print(total)
```

Trace it on paper: `n` and `total` after every pass.

Item 1 · Answer

```
shares = [10, 4, 2]
total = 0
for n in shares:
    if n > 3:
        total += n
print(total)
```

```
14
```

pass	`n`	`n > 3`	`total`
1	10	True	10
2	4	True	14
3	2	False	14

growth.py

```python
# growth.py — session 2's position.py, with one line changed (review item)
ticker = "NESN"
raw = "4"              # text, as read from a file
shares = int(raw)
price = 102.3
cash = 1000.0
last_sale = None
print(raw * 2, shares * 2, type(raw), type(shares))

shares = shares + 6    # buy 6 more
cash -= 6 * price
value = shares * price
weight = value / (value + cash)
can_buy = int(cash // price)
rest = cash % price
in_2y = value * 1 + 0.05 ** 2

valid = len(ticker) <= 5 and shares > 0
alert = weight > 0.5 or not valid

print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
print(f"cash {cash:.2f}, weight {weight:.2f}")
print(can_buy, rest, round(in_2y, 2))
print(valid, alert, last_sale)
```

```
$ python growth.py
44 8 <class 'str'> <class 'int'>
386.20000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.30000000000005 1023.0
True True None
```

Item 2 · Is this output plausible?

`value` is 1023.0 CHF. `in_2y` should be the value after **two years at 5% a year**.

The last number on output line 5 is `1023.0`.

Plausible? If not, which line is the bug, and what kind of failure is it?

growth.py

```python
# growth.py — session 2's position.py, with one line changed (review item)
ticker = "NESN"
raw = "4"              # text, as read from a file
shares = int(raw)
price = 102.3
cash = 1000.0
last_sale = None
print(raw * 2, shares * 2, type(raw), type(shares))

shares = shares + 6   # buy 6 more
cash -= 6 * price
value = shares * price
weight = value / (value + cash)
can_buy = int(cash // price)
rest = cash % price
in_2y = value * 1 + 0.05 ** 2
        ** first, then *, then +
valid = len(ticker) <= 5 and shares > 0
alert = weight > 0.5 or not valid

print(cash, cash == 386.2, abs(cash - 386.2) < 0.01)
print(f"{ticker}: {shares} shares, {value:,.2f} CHF")
print(f"cash {cash:.2f}, weight {weight:.2f}")
print(can_buy, rest, round(in_2y, 2))
print(valid, alert, last_sale)
```

```
$ python growth.py
44 8 <class 'str'> <class 'int'>
386.20000000000005 False True
NESN: 10 shares, 1,023.00 CHF
cash 386.20, weight 0.73
3 79.30000000000005 1023.0
True True None
```

Item 2 · Answer

Not plausible. Rough check: 5% of 1,000 is 50 a year, so expect about **1,100** — not the same number.

Line 16 computes `0.05 ** 2` first (0.0025), then `value * 1`, then adds. It should be `value * 1.05 ** 2`.

No error: **kind 3, it runs and lies.**

order.py

```python
# order.py — an office-supplies order, with one planted lie (review item 3)
prices = {"pen": 1.5, "pad": 3.0, "ink": 4.0}
qty = {"pen": 4, "pad": 2, "ink": 1}

total = 0
for item in prices:
    total = total + prices[item] * qty["pen"]

print(f"order total: {total:.2f} CHF")
```

```
$ python order.py
order total: 34.00 CHF
```

Item 3 · Which line does not follow the brief?

Brief: the order total is, for every item, its price times the quantity ordered.

The script runs without an error and prints the line on the right.

Which line does not follow the brief? Check one item by hand first.

order.py

```
1  # order.py — an office-supplies order, with one planted lie (review item 3)
2  prices = {"pen": 1.5, "pad": 3.0, "ink": 4.0}
3  qty = {"pen": 4, "pad": 2, "ink": 1}
4
5  total = 0
6  for item in prices:
7      total = total + prices[item] * qty["pen"]
8
9  print(f"order total: {total:.2f} CHF")
```

always the pens' quantity

```
$ python order.py
order total: 34.00 CHF
```

Item 3 · Answer

Line 7. It multiplies every price by `qty["pen"]`, the quantity of pens, on every pass. It should be `qty[item]`.

By hand: 1.5 × 4 + 3.0 × 2 + 4.0 × 1 = **16.00**, not 34.00.

No error: **kind 3, it runs and lies.**

Item 4 · Which kind of failure?

The comment says what the programmer wanted.

(a)

```
print("total", 12   # wanted: total 12
```

(b)

```
code = 42
print(len(code))   # wanted: 2 digits
```

(c)

```
print(round(2.5))   # wanted: 3
```

Won't start, crashes, or runs and lies? If it prints, say what.

Item 4 · Answer

(a) won't start

```
print("total", 12   # wanted: total 12
```

```
SyntaxError: '(' was never closed
```

(b) crashes

```
code = 42
print(len(code))   # wanted: 2 digits
```

```
TypeError: object of type 'int' has no len()
```

(c) runs and lies

```
print(round(2.5))   # wanted: 3
```

```
2
```

(a) a bracket is never closed: nothing runs. (b) a number has no length: it stops with a message. (c) `round` sends a tie to the **even** number (session 1): **2, and no message at all**.

Item 5 · Calls: positional, keyword, default

```
def shipping(weight, rate=2, minimum=5):
    cost = weight * rate
    if cost < minimum:
        cost = minimum
    return cost
```

What does each call give? If a line stops Python before anything runs, say **won't start**; if it stops while running, say **crashes**.

(a) `shipping(1)` · **(b)** `shipping(4)` · **(c)** `shipping(4, minimum=10)` · **(d)** `shipping(rate=3, weight=4)` · **(e)** `shipping(rate=3, 4)` · **(f)** `shipping()`

Item 5 · Answer

```
def shipping(weight, rate=2, minimum=5):
    cost = weight * rate
    if cost < minimum:
        cost = minimum
    return cost

print(shipping(1), shipping(4), shipping(4, minimum=10), shipping(rate=3, weight=4))
```

```
5 8 10 12
```

(a) 2 is below the minimum: **5**. (b) **8**. (c) 8 is below the new minimum: **10**. (d) keywords, any order: **12**.

(e) **won't start**: a positional argument after a keyword. (f) **crashes**: `weight` has no default.

Item 6 · What appears on the screen?

```
def with_tax(amount):
    print(amount * 0.1)
    return amount

total = with_tax(100) + with_tax(50)
print(total)
```

The brief: `with_tax` gives back the amount **with 10% tax added**. Write every line the screen shows. Does the program follow the brief?

Item 6 · Answer

```python
def with_tax(amount):
    print(amount * 0.1)
    return amount

total = with_tax(100) + with_tax(50)
print(total)
```

```
10.0
5.0
150
```

Each call **prints** the tax, then **returns** the amount unchanged. The two calls run first, then the last `print`.

`150` is the amount without tax: **runs and lies**. Line 3 should be `return amount * 1.1` (165.0, give or take a float).

Item 7 · Nested data, one step at a time

```
fleet = {"van1": {"km": [120, 80, 95], "driver": "Ana"},
         "van2": {"km": [60, 140], "driver": "Ben"}}
```

What does each expression give? Say **crashes** if it stops the program.

(a) `fleet["van1"]["km"][-1]` · **(b)** `sum(fleet["van2"]["km"])` · **(c)** `fleet["van2"]["driver"][0]` · **(d)** `fleet["van1"]["km"][1:]` · **(e)** `fleet["van1"]["km"][3]` · **(f)** `fleet["van1"][0]`

At each bracket ask: **what type do I have now?**

Item 7 · Answer

```
fleet = {"van1": {"km": [120, 80, 95], "driver": "Ana"},
         "van2": {"km": [60, 140], "driver": "Ben"}}
print(fleet["van1"]["km"][-1], sum(fleet["van2"]["km"]),
      fleet["van2"]["driver"][0], fleet["van1"]["km"][1:])
```

```
95 200 B [80, 95]
```

(a)–(d) as printed: a number, a number, **a one-letter string**, **a list**.

(e) **crashes**: the list has positions 0, 1, 2 only. (f) **crashes**: a dict has no positions, only keys.

Item 8 · Decisions in a loop

```
ages = [17, 25, 30, 70]
students = [False, True, False, False]
for i in range(len(ages)):
    if ages[i] < 18 or students[i]:
        print("reduced")
    elif ages[i] >= 65:
        print("senior")
    else:
        print("full")
```

What does it print? Then: what would the 70-year-old get if the `or` were an `and`?

Item 8 · Answer

```
ages = [17, 25, 30, 70]
students = [False, True, False, False]
for i in range(len(ages)):
    if ages[i] < 18 or students[i]:
        print("reduced")
    elif ages[i] >= 65:
        print("senior")
    else:
        print("full")
```

```
reduced
reduced
full
senior
```

17: under 18. 25: a student. 30: neither, not 65: full. 70: not reduced, so the `elif` decides.

With `and`, nobody here is both under 18 **and** a student: the first two become **full**; the 70-year-old is still **senior**.

Item 9 · Fill the gaps

```
names = ["Bern", "Basel", "Chur"]
orders = [310, 480, 95]
best = [ 1 ]
best_name = names[0]
for i in range(len(orders)):
    if orders[i] [ 2 ] best:
        best = orders[i]
        best_name = [ 3 ]
print(f"largest: {best} ({best_name})")
```

The program must print `largest: 480 (Basel)`.

Gap 1: `orders[0]` · `names[0]` · `orders` · `None`

Gap 2: `>` · `<` · `==`

Gap 3: `names[i]` · `names[0]` · `i` · `orders[i]`

Item 9 · Answer

```
names = ["Bern", "Basel", "Chur"]
orders = [310, 480, 95]
best = orders[0]
best_name = names[0]
for i in range(len(orders)):
    if orders[i] > best:
        best = orders[i]
        best_name = names[i]
print(f"largest: {best} ({best_name})")
```

```
largest: 480 (Basel)
```

Gap 1: start from a real order, not a name, a list or `None` (comparing a number with those crashes). Gap 2: `<` would find the **smallest**. Gap 3: the name **at the same position**, `names[i]`; `i` would print `1`.

Item 10 · What does this function do?

```
def f(prices, budget):
    spent = 0
    n = 0
    for p in prices:
        if spent + p <= budget:
            spent = spent + p
            n = n + 1
    return n
```

Which sentence describes `f` best?

a. The number of prices that are at most the budget.
b. The total spent.
c. How many items you can buy, in order, skipping any item that would go over the budget.
d. The first price that goes over the budget.

Item 10 · Answer

c. Test it on numbers you can follow:

```
def f(prices, budget):
    spent = 0
    n = 0
    for p in prices:
        if spent + p <= budget:
            spent = spent + p
            n = n + 1
    return n

print(f([5, 8, 3, 6], 12))
```

```
2
```

5 fits (spent 5). 8 would make 13: skipped. 3 fits (spent 8). 6 would make 14: skipped. So **2**. Sentence **a** would give 4; **b** would give 8.

Say the function in one sentence, then test the sentence on a small example.

regions.py

```python
# regions.py — sales by region (review item 11)
def parse(line):
    region, amount = line.split(";")
    return region, float(amount)

def totals_by_region(lines):
    totals = {}
    for line in lines:
        region, amount = parse(line)
        if region in totals:
            totals[region] = totals[region] + amount
        else:
            totals[region] = amount
    return totals

lines = ["west;120.5", "east;80", "west;40", "north;15.5", "east;20"]
result = totals_by_region(lines)
for region in result:
    print(region, result[region])
print(len(result), "regions")
```

```
$ python regions.py
not run yet
```

Item 11 · Read the whole program

Read it once, top to bottom, before you answer.

1. Which line handles a region seen **for the first time**?
2. What is the **first** line printed?
3. What is the **last** line printed?
4. Lines 10–13 are replaced by one line: `totals[region] = amount`. What does `west` show now?

regions.py

```python
# regions.py — sales by region (review item 11)
def parse(line):
    region, amount = line.split(";")
    return region, float(amount)

def totals_by_region(lines):
    totals = {}
    for line in lines:
        region, amount = parse(line)
        if region in totals:
            totals[region] = totals[region] + amount
        else:
            totals[region] = amount   # a new key
    return totals

lines = ["west;120.5", "east;80", "west;40", "north;15.5", "east;20"]
result = totals_by_region(lines)
for region in result:
    print(region, result[region])
print(len(result), "regions")
```

```
$ python regions.py
west 160.5
east 100.0
north 15.5
3 regions
```

Item 11 · Answer

1. **Line 13**: the `else` branch, when the region is not yet a key.
2. `west 160.5` : 120.5 + 40. A dict keeps keys **in the order they were added**.
3. `3 regions` : west, east, north. A key appears only once.
4. **`west 40.0`** : each line now **replaces** the value instead of adding to it. The last west line wins. Runs and lies.

Item 12 · Only one output is possible

Brief: read the file, **skip** students with no score, and print how many scores were used and their average.

```
name,score
Ana,6
Ben,
Chloé,4
Dan,5
```

Four programs printed four lines. Only one follows the brief. Which one? Do it by hand, no code needed.

a. `4 scores, average 3.75` · **b.** `3 scores, average 15.0` · **c.** `3 scores, average 5.0` · **d.** `3 scores, average 6.0`

Item 12 · Answer

c. Ana, Chloé, Dan: (6 + 4 + 5) / 3 = **5.0**.

```
scores = [6, 4, 5]          # Ben is skipped: no score
print(len(scores), "scores, average", sum(scores) / len(scores))
```

```
3 scores, average 5.0
```

- **a.** Ben was counted as 0: **missing is not zero**.
- **b.** The sum, never divided.
- **d.** Only the first score: a `return` or a reset in the wrong place.

Item 13 · Bonus-style: one-line code

```
pairs = [("pen", 1.5), ("ink", 4.0), ("pad", 3.0)]
cheap = [name for name, price in pairs if price < 3.5]
print(cheap)
print(max(pairs, key=lambda p: p[1])[0])
```

What does it print? Say each line in one English sentence first.

Item 13 · Answer

```
pairs = [("pen", 1.5), ("ink", 4.0), ("pad", 3.0)]
cheap = [name for name, price in pairs if price < 3.5]
print(cheap)
print(max(pairs, key=lambda p: p[1])[0])
```

```
['pen', 'pad']
ink
```

Line 2: "the names of the items that cost less than 3.5", in their original order.

Line 4: "the pair with the largest **second** element, then its first part". Without `key=`, `max` would compare the names as text and give `pen`.

Before the midterm

- **Trace on paper.** Every script in this part: cover the output, predict, then check.
- **Name the failure:** won't start, crashes, or runs and lies?
- **Check one number by hand.** It catches the runs-and-lies kind, which no machine will.
- **Say each function in one sentence**, then test the sentence on a small example.
- **Read the whole question first**, and the brief if there is one.

Not on the midterm: git · reading tracebacks · writing classes · **extra** slides.

Next session: the midterm, Monday 2 November. Bring your laptop, charged.